

2025 – 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY: INFORMATION TECHNOLOGY

Design of a Multi-Agent AI Pipeline
for Evidence Collection from
Illegal Streaming Sites

By: Ahmed ARFAOUI

Academic supervisor: [Academic Supervisor]

Corporate Internship Supervisor: Wael Hkiri

LOGO ENTREPRISE

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- **Nom & Prénom / Name & Surname :** Ahmed ARFAOUI

Encadrant Entreprise / Business site Supervisor

- **Nom & Prénom / Name & Surname :** Wael Hkiri

Cachet & Signature / Stamp & Signature

Encadrant Académique / Academic Supervisor

- **Nom & Prénom / Name & Surname :** [Academic Supervisor]

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné** / This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde / This form must be inserted after the cover page.

Abstract

This report presents the design, development, and evaluation of an agentic evidence-collection system for illegal streaming websites. The work was carried out during an internship at Soft Stars for the client beIN SPORTS. The objective was to support rights-enforcement workflows by visiting target websites, classifying their pages, extracting hosting and stream evidence, and preserving auditable outputs such as screenshots, traces, and structured results.

The internship produced two related artifacts: a validated **n8n** workflow, from which one operational agent was deployed in the company context, and a **Python**-based reference architecture designed to improve maintainability, observability, memory, evaluation, and future scalability. The company-side operational outcome therefore remained a focused single-agent workflow, while the broader FastAPI/Puppeteer multi-agent implementation is presented as an academic and engineering reference architecture derived from the validated n8n workflow.

The proposed architecture organizes evidence collection around specialized reasoning agents, browser automation tools, structured outputs, trace storage, screenshot capture, and evaluation metrics. Its target deployment model separates API services, browser workers, observability storage, screenshot storage, and queue-based execution for future Azure Container Apps Jobs operation. The Python architecture does not depend on the enterprise production database; the database described in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs.

Acknowledgements

I would like to thank my academic and professional supervisors for their guidance throughout this internship. I also thank the Soft Stars team for the technical discussions, feedback, and trust that made this work possible.

Contents

1	Introduction	1
1.1	Host organization – Soft Stars	1
1.2	Sports piracy and the evidence collection challenge	1
1.3	Objectives and contributions	2
1.4	Report structure	3
2	Methodology, Requirements Analysis, and Design Constraints	4
2.1	Methodology options considered	4
2.1.1	Agile	4
2.1.2	CRISP-DM	5
2.1.3	Design Science Research Methodology (DSRM)	5
2.2	Chosen methodology: DSRM with iterative prototyping	5
2.3	DSRM applied to this project	6
2.4	Project evolution timeline	7
2.5	Functional and non-functional requirements	8
2.6	Design constraints	10
2.7	Prompting methodology evolution: intent-based to ReAct-style loops	11
2.8	Derived design principles	12
3	Technical Background	13
3.1	Streaming Website Structures	13
3.1.1	The normal page flow	13

3.1.2	Landing pages	14
3.1.3	Hosting pages	14
3.1.4	Embedded pages	15
3.2	DOM Structures and Web Page Representation	16
3.2.1	A simplified DOM tree	16
3.2.2	DOM signals on streaming pages	16
3.3	Streaming Protocols and Embedded Players	17
3.3.1	Common embedding patterns	17
3.3.2	HLS and MPEG-DASH	18
3.3.3	Tokens, cookies, referers, and provider checks	19
3.4	Browser Automation, Scraping, and Runtime Obstacles	19
3.4.1	Puppeteer as a browser control layer	19
3.4.2	Ads, cookies, popups, and other annoyances	20
3.4.3	Why screenshots and network logs matter	20
3.5	Agentic Evidence Collection Concepts	21
3.5.1	Agents and ReAct-style loops	21
3.5.2	Prompt engineering, tokens, and token usage	21
3.5.3	n8n and workflow automation	22
3.5.4	LangChain, tools, and MCP servers	22
3.6	Chapter Summary	23
4	System Architecture	24
4.1	System Overview and Use Cases	24
4.2	Logical Agent Architecture	25
4.2.1	Run Sequence	27
4.3	Static Code and Data Model View	27
4.4	Physical Runtime and Supporting Services	29
4.4.1	Validation and Repair Loop	29
4.5	Memory, Caching, and Deployment Summary	30

4.6	Chapter Summary	30
5	Implementation	31
5.1	Part 1: n8n Workflow Prototype	31
5.1.1	Purpose of the n8n phase	31
5.1.2	n8n agents and screenshot placeholders	32
5.1.3	Limitations of the n8n implementation	33
5.2	Part 2: Implemented Reference Solution	35
5.2.1	FastAPI Backend	35
5.2.2	Next.js Operator Console	36
5.2.3	MCP Browser Containers and Puppeteer Tools	37
5.2.4	Agents, Models, Memory, and Prompts	38
5.2.5	PostgreSQL Database and Observability	41
5.3	Implementation Summary	42
6	Evaluation and Testing	44
6.1	Evaluation Sources	44
6.2	Dashboard Snapshot	45
6.3	KPI Definitions and Formulas	46
6.4	Cost Estimation Formula	47
6.5	Manual Evaluation Protocol	48
6.6	Evaluation by Use Case and Agent	49
6.7	Prompt Regression Evaluation	49
6.8	Manual Reference-Set Results	50
6.9	Tool Evaluation	51
6.10	LLM Model Evaluation	52
6.11	Cache and Pricing Evaluation	53
6.12	Hard-Case Evaluation	54
6.13	Chapter Conclusion	55

7	Conclusion and Perspectives	56
7.1	General Conclusion	56
7.2	Main Contributions	57
7.3	Technical Lessons	57
7.3.1	Prompt structure matters	57
7.3.2	Browser evidence must be layered	57
7.3.3	Fallbacks are part of the architecture	58
7.3.4	Evaluation must be regression-oriented	58
7.4	Limitations	58
7.5	Future Work	58
7.5.1	Screenshot annotation layer	59
7.6	Final Remarks	59
	References	60

CHAPTER 1

Introduction

1.1 Host organization – Soft Stars

Soft Stars is a software company working in digital media intelligence, content protection, and technical support for rights-enforcement operations. Its work focuses on helping media organizations observe online distribution channels, identify unauthorized use of protected content, and prepare technical evidence that can be reviewed by operational, legal, or business teams.

The internship was carried out within Soft Stars in the context of sports media protection. The project was connected to the broader problem of monitoring illegal streaming websites and improving the way evidence is collected from them. At this stage of the report, the system is introduced only from a general business and engineering perspective. The detailed process, implementation choices, and evaluation are developed progressively in the following chapters.

1.2 Sports piracy and the evidence collection challenge

Premium sports broadcasting depends on exclusive distribution rights. Broadcasters pay for the right to transmit competitions in specific territories, on specific platforms, and under controlled commercial conditions. Illegal streaming websites undermine this model by redistributing protected events without authorization, often through domains that change frequently and through pages designed to be difficult to monitor manually.

The first practical challenge in this context is not only finding suspicious websites. It is collecting evidence that is precise, traceable, and useful. A rights holder needs to know

which page was reachable, when it was observed, what content or player it exposed, and which technical indicators can support a later review or enforcement action. Screenshots, visible page state, embedded players, network indicators, and stream references can all become important, but they are often hidden behind dynamic interfaces, popups, nested frames, language variation, and short-lived links.

This makes evidence collection an engineering problem. Manual review is slow and inconsistent when the number of targets grows. Simple scraping is too fragile for modern streaming pages, because the useful evidence may appear only after the page loads scripts, reacts to user actions, or redirects through several layers. The general goal is therefore to support a more reliable way of observing these websites, collecting evidence, and keeping enough context for the result to be auditable.

1.3 Objectives and contributions

The objective of this project is to study and design a technical approach for evidence collection from illegal sports streaming websites. The work aims to move from a manual and case-by-case process toward a more structured process that can inspect target pages, preserve useful observations, and present results in a form that can be reviewed.

The main objectives are:

- understand the operational context of sports piracy monitoring and the type of evidence required by rights-protection teams;
- analyze the technical behavior of illegal streaming websites, including dynamic pages, embedded players, redirects, and multilingual layouts;
- define a maintainable evidence-collection process that can adapt to different website structures;
- design a system architecture that separates observation, extraction, persistence, review, and evaluation concerns;
- evaluate the approach on representative targets and document its limits.

The main contributions of the internship are:

- a structured analysis of the evidence-collection problem in the sports piracy domain;
- a progressive design of an automated collection approach, from general requirements to implementation details;
- a technical architecture for collecting, storing, and reviewing evidence from suspicious streaming websites;
- an evaluation of the approach based on observed behavior, collected traces, and failure cases;

- a report structure that moves from the general business problem toward the specialized implementation and validation work.

1.4 Report structure

This report is organized from general context to specialized implementation. Chapter 2 presents the methodology, requirements analysis, and design constraints used to frame the project. Chapter 3 introduces the technical background needed to understand streaming websites, browser-based observation, automation tools, agentic extraction, and media protocols. Chapter 4 describes the proposed architecture and explains how the main system responsibilities are organized. Chapter 5 details the implementation choices and the concrete mechanisms used to collect and structure evidence. Chapter 6 evaluates the results, discusses observed limitations, and analyzes the reliability of the approach. Chapter 7 concludes the report.

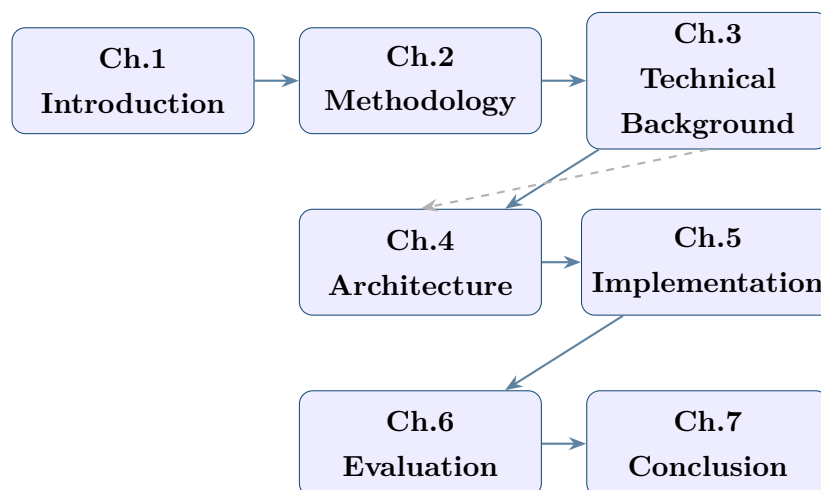


Figure 1.1: Report reading map: chapter sequence from general context to implementation and evaluation.

CHAPTER 2

Methodology, Requirements Analysis, and Design Constraints

This chapter presents the methodological frame adopted for the internship, explains why Design Science Research Methodology (DSRM) was selected, maps the six DSRM phases to the concrete work carried out, and derives the requirements and constraints that shaped the artifact described in Chapters 4 and 5. Because the internship delivered and evaluated technical artifacts rather than a predictive model, the methodological emphasis is on artifact design, demonstration, and evaluation.

2.1 Methodology options considered

The internship combined software engineering, browser automation, prompt design, and evidence-oriented investigation. Three methodological options were considered: Agile, CRISP-DM, and Design Science Research Methodology (DSRM).

2.1.1 Agile

Agile practices matched the day-to-day execution rhythm of the internship. Short cycles of implementation, run inspection, failure analysis, and prompt or tool revision were essential because illegal streaming sites change quickly and often expose new obstacles only during live testing.

2.1.2 CRISP-DM

CRISP-DM [13] offers a clear sequence from business understanding to deployment and evaluation. Parts of that logic were useful for organizing observations and reporting progress. However, the project was not a classical data-mining lifecycle and did not aim to optimize a predictive model on a labeled dataset.

2.1.3 Design Science Research Methodology (DSRM)

Design science research focuses on the creation and evaluation of artifacts that solve relevant organizational problems [14, 15]. This framing matches the internship more closely because the central deliverable is an engineering artifact: an agentic browser-based evidence-collection pipeline composed of prompts, tools, browser traces, screenshots, structured outputs, and evaluation metrics.

Methodology	Strength for this project	Limitation for this project
Agile	Excellent short-cycle execution and adaptation during prompt, tool, and browser refinement	Less explicit about how to justify and evaluate a technical artifact as a research contribution
CRISP-DM	Useful discipline for scoping, evaluation, and deployment discussion	Dataset/model-centered framing is not the main logic of this internship
DSRM	Artifact-centered framing that explicitly links problem, design, demonstration, and evaluation	Needs complementary iteration rhythm for fast engineering feedback

Table 2.1: Methodology comparison for the internship project.

2.2 Chosen methodology: DSRM with iterative prototyping

DSRM was adopted as the overall methodology, while Agile-style short iterations were used inside each phase to refine prompts, tools, and browser behavior based on observed failures.

DSRM fits the project better than CRISP-DM for three reasons. First, the project is artifact-centered, not dataset-centered or model-centered: the main work was to design a reliable evidence-collection system under adversarial web conditions. Second, the main

output is a working and proposed system architecture together with a validated agent workflow, not a trained predictive model. Third, evaluation focuses on tool behavior, evidence quality, trace completeness, extraction success and failure patterns, cost, and robustness rather than on statistical accuracy against a fixed labeled benchmark.

CRISP-DM remains a useful secondary lens for discussing operational context and evaluation discipline, but it is not the primary methodological backbone of the report. Figure 2.1 summarizes the adopted DSRM view.

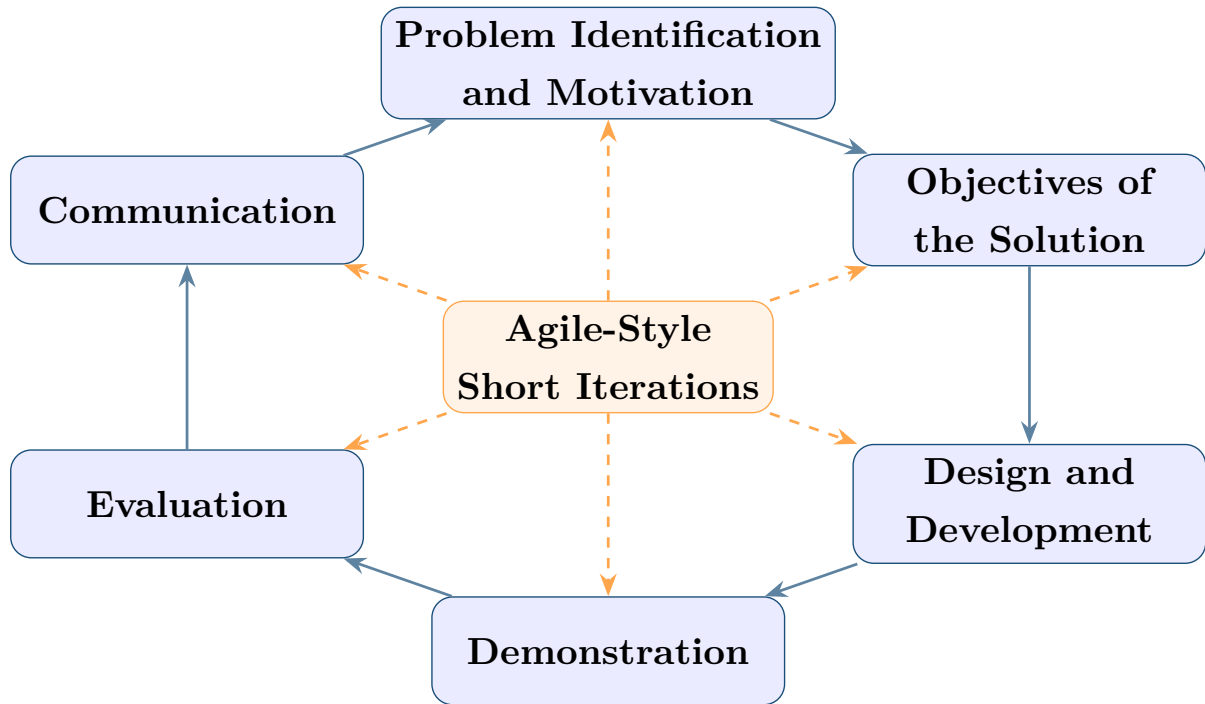


Figure 2.1: DSRM adopted for the project, with Agile-style short iterations inside each phase.

2.3 DSRM applied to this project

Following Peffers et al. [15], the project was organized around six DSRM phases. Table 2.2 maps each phase to the concrete internship work and its main outputs.

DSRM phase	Application in this project	Main output
Problem identification and motivation	Illegal streaming evidence collection is dynamic, adversarial, multilingual, and difficult for static scrapers because evidence may appear only after runtime interaction, iframe traversal, or player activation.	Problem framing and motivation for the artifact
Objectives of the solution	Define that the artifact must classify pages, extract host/embed/stream evidence, capture screenshots, preserve traces, and calculate evaluation metrics that support reviewable evidence collection.	Solution objectives and acceptance criteria
Design and development	Build the n8n prototype and the validated single-agent workflow used in the company context; derive a Python/FastAPI reference architecture for modularity, observability, memory, evaluation, and future scalability.	Operational workflow plus reference architecture
Demonstration	Run the artifact on representative streaming-site families: landing pages, hosting pages, embedded player pages, and iframe/popup/tokenized-stream cases.	Demonstration cases and observed behaviors
Evaluation	Measure prompt behavior, tool usage, extraction success and failure, token and cost metrics, cache behavior, trace quality, and practical limitations.	Evaluation traces, metrics, and findings
Communication	Produce the internship report, diagrams, architecture documentation, and a future deployment roadmap for industrialization.	Communicated artifact and future roadmap

Table 2.2: Application of DSRM to the evidence-collection project.

2.4 Project evolution timeline

The internship evolved from early n8n validation to a company-side single-agent deployment, then to the Python reference architecture, evaluation campaigns, and a future target cloud deployment. Figure 2.2 summarizes this sequence.

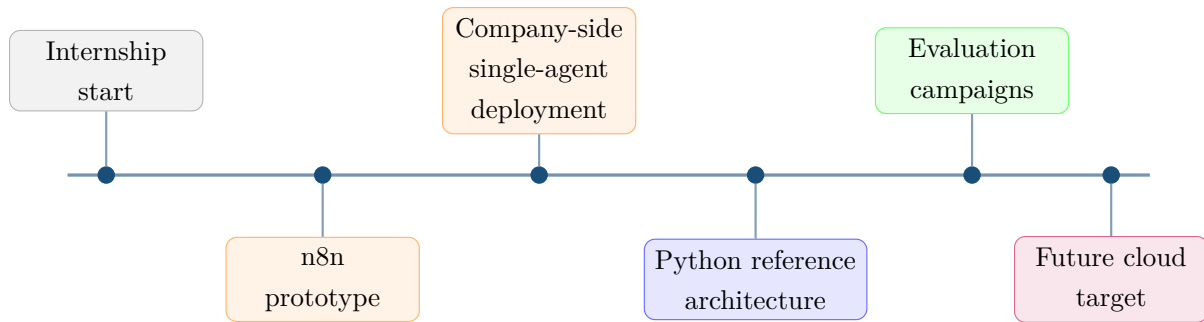


Figure 2.2: Project evolution timeline from n8n validation to Python reference architecture and future deployment target.

The validated n8n workflow established the operational baseline used in the company context. The Python reference architecture should be interpreted as the maintainability-oriented evolution of the same evidence-collection logic, while the future cloud target remains a planned deployment direction rather than a claim of full platform replacement during the internship.

2.5 Functional and non-functional requirements

The requirements were addressed across two complementary levels. At the operational level, the validated n8n workflow covered the core evidence-collection tasks used in the company context. At the reference-architecture level, the Python implementation generalized those tasks into a more maintainable design with stronger observability, memory, evaluation support, and future scalability.

Table 2.3 summarizes the ten functional requirements derived from the operational context.

ID	Requirement summary	Priority
FR-1	URL ingestion and run lifecycle management	Critical
FR-2	Page-type classification with DOM and screenshot signals	Critical
FR-3	Specialist routing by page type	Critical
FR-4	Tab-aware landing-page candidate discovery	High
FR-5	Multi-server hosting extraction loop	Critical
FR-6	Embedded player activation and stream capture	Critical
FR-7	Hosting-to-embedded fallback	High
FR-8	Screenshot capture and persistent evidence storage	High
FR-9	WHOIS / IP infrastructure enrichment	Medium
FR-10	Operator review surface and trace inspection	High

Table 2.3: Functional requirements summary with priority.

Table 2.4 lists the non-functional requirements and the design choices used to address them across the validated workflow and the Python reference architecture.

ID	Non-functional requirement	How it is addressed
NFR-1	Maintainability	n8n validated behavior; Python modular agents, versioned prompts, and tool registry
NFR-2	Observability	Structured event log, evaluation database, SSE review surface, and trace persistence
NFR-3	Cost control	Model routing, provider-side caching, and token budgets
NFR-4	Reliability	Tool error contracts, popup handling, and bounded retry logic
NFR-5	Scalability	Target Azure Container Apps Jobs execution with Azure Service Bus queueing
NFR-6	Portability	Docker Compose, configurable browser profiles, and separable worker services
NFR-7	Evidence traceability	Run-linked artifact chain: event → screenshot → structured output

Table 2.4: Non-functional requirements and implementation approach.

2.6 Design constraints

From a design science perspective, the artifact had to satisfy the problem environment while remaining evaluable and maintainable. Six constraints bounded the design space:

1. target-site layouts and anti-bot behavior can change without notice between broadcast windows;
2. the player can be nested behind multiple iframe hops from the top-level page;
3. popups, overlays, and fake play controls can divert scripted extraction;

4. final stream manifests may only be generated after a runtime token exchange triggered by player activation;
5. inference cost scales with exploration depth, the number of hosting pages, and the number of servers tested per page, requiring active caching and model routing;
6. the Python reference architecture must remain decoupled from enterprise production databases, so persistence in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs.

2.7 Prompting methodology evolution: intent-based to ReAct-style loops

The prompting strategy evolved during the DSRM design-and-development and evaluation phases. The first version relied on high-level intent prompts, which often produced premature answers on ambiguous pages. The later version adopted a ReAct-style loop in which the agent had to observe, reason, act, and re-check the browser state before declaring success. Figure 2.3 compares the two approaches side by side.

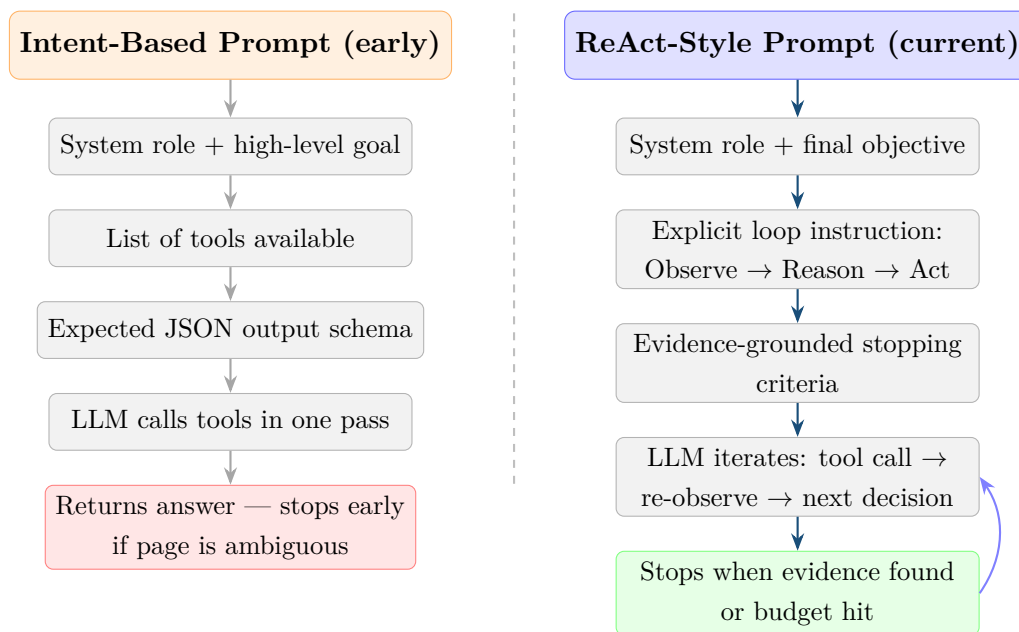


Figure 2.3: Prompt strategy evolution: intent-based linear flow versus ReAct-style iterative loop.

Why ReAct performed better. ReAct-style behavior:

- reduced early stopping on ambiguous pages where the first observation was insufficient;

- enforced evidence-driven step progression, so the agent could not declare success without a browser-grounded confirmation step;
- made tool traces easier to audit because each reasoning step was explicitly logged before and after the action.

2.8 Derived design principles

The methodology, constraints, and evaluation results led to the following design principles:

- classify page role before deep extraction;
- separate operational workflow validation from reference-architecture generalization;
- ground every conclusion in browser evidence rather than textual guesswork;
- preserve auditable traces, screenshots, and structured outputs for every critical step;
- use specialist agents and explicit fallback paths rather than one unconstrained agent;
- optimize robustness together with cost through caching, budgets, and model routing;
- design the future cloud target around loose coupling between API services, browser workers, observability storage, and artifact storage.

Taken together, the DSRM framing, the two-level requirement view, and the derived design principles establish the scope of the artifact presented in this report. The next chapter establishes the technical background needed to understand the website structures, browser tools, streaming protocols, and agentic concepts used by the artifact. Chapter 4 then translates the methodology and technical background into a concrete reference architecture.

CHAPTER 3

Technical Background

This chapter introduces the technical concepts needed before presenting the system architecture and implementation. It deliberately stays at a general level: the goal is to explain the web structures, streaming mechanisms, browser automation tools, and agentic concepts that make the project understandable. The concrete architecture, database schema, prompts, and tool contracts are described later.

The chapter moves from the target websites themselves toward the technologies used to analyze them. It starts with page structures and DOM representation, then explains streaming protocols and embedded players, then covers browser automation issues such as ads, cookies, popups, and anti-bot providers. It ends with the agentic concepts used in the project: agents, prompts, tool use, n8n, LangChain, MCP servers, model providers, caching, and token usage.

3.1 Streaming Website Structures

Illegal streaming websites are not a single page type. They are usually a chain of pages with different responsibilities: one page lists events, another page presents a selected match and server choices, and a final embedded page or iframe hosts the player. Understanding these page roles is necessary before discussing extraction tools, because each role exposes different evidence and requires different interaction.

3.1.1 The normal page flow

Most target websites can be described as a three-step navigation path. A landing page contains match listings. A hosting page focuses on one selected match and usually offers one or more streaming servers. An embedded page contains the actual player, often inside

an iframe loaded from another domain.

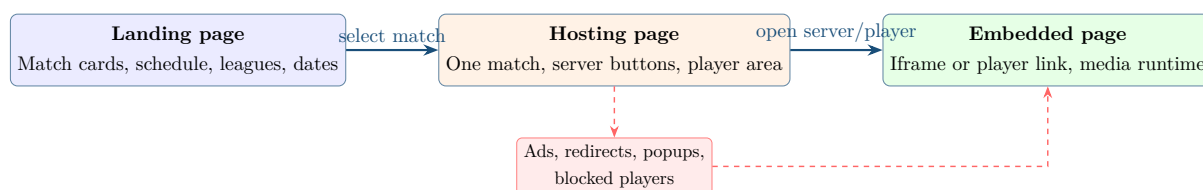


Figure 3.1: Common navigation flow from event listing to hosting page and embedded player.

This flow is not always clean. Some sites skip the hosting page and link directly to an embedded player. Others place several ad pages between the hosting page and the player. Still, this model is useful because it separates discovery, server selection, and media extraction into distinct technical problems.

3.1.2 Landing pages

Landing pages are the discovery layer. They usually contain match cards, schedule rows, league filters, date tabs, search boxes, or category menus. The useful output from a landing page is not the stream itself but a set of candidate hosting links and match metadata.

Common landing-page layouts include:

- **card grids:** repeated blocks with teams, time, league, and a watch link;
- **tables or lists:** one row per event, often with a direct action link;
- **tabbed schedules:** live, today, yesterday, tomorrow, or league-specific tabs loaded through JavaScript;
- **lazy-loaded sections:** match cards that appear only after scrolling or after an XHR request;
- **embedded JSON:** page data stored in `<script type="application/json">` blocks or `data-*` attributes.

The important idea is that landing pages are structurally repetitive. Even when class names change, the page still tends to contain repeated event units. This makes layout recognition more useful than relying on one fixed CSS selector.

3.1.3 Hosting pages

Hosting pages are the decision layer. They normally focus on one match and contain a player container, a server list, tabs such as **Server 1** or **HD**, and sometimes channel names

or broadcaster logos. A hosting page may already expose the stream URL, but more often it only exposes an iframe or a JavaScript player configuration.

Typical hosting-page signals are:

- one match title or two team names repeated near the player;
- server buttons, mirrors, or quality selectors;
- an iframe whose `src` points to a player page;
- player libraries such as Video.js, JWPlayer, Clappr, hls.js, or dash.js;
- visible state changes after clicking play, selecting a server, or closing an overlay.

Hosting pages are more difficult than landing pages because they require interaction. The same click can activate a real player, open a popup, switch a server, or trigger a redirect. This is why browser automation and runtime observation become necessary.

3.1.4 Embedded pages

Embedded pages are the media layer. They are often minimal pages loaded inside iframes, with little content except a player, a play button, and scripts that request the final media manifest. They may belong to another domain, which means the top page and the player page have different DOMs, cookies, and network contexts.

An embedded page is important because the final stream reference is often generated only there. A static request to the hosting page may not show any `.m3u8` or `.mpd` URL, while the embedded player requests the manifest after user activation.

Page type	Main role	Useful evidence
landing_page	Discover events and candidate match pages	match links, team names, schedule metadata, listing screenshot
hosting_page	Select server and activate player	server labels, iframe URLs, player state, page screenshot
embedded_page	Load the final media runtime	stream manifest, segment requests, player diagnostics
other	Filter noise such as ads, redirects, and unrelated pages	reason for skip or blocked state

Table 3.1: General page taxonomy used to reason about streaming-site structure.

The page taxonomy turns a messy website into a sequence of smaller questions: Where are the matches? Which page hosts the selected match? Which server or iframe contains the player? Which runtime request proves that media was loaded?

3.2 DOM Structures and Web Page Representation

The Document Object Model (DOM) is the browser's tree representation of a web page. It turns HTML into nodes such as documents, elements, attributes, text nodes, iframes, buttons, images, and scripts. Browser automation tools operate on this tree when they query selectors, read attributes, click buttons, or remove overlays.

3.2.1 A simplified DOM tree

The diagram below shows a simplified DOM structure. It is intentionally basic, but it captures the idea used throughout the project: a visible page is not just text, it is a tree of nested elements with attributes and event handlers.

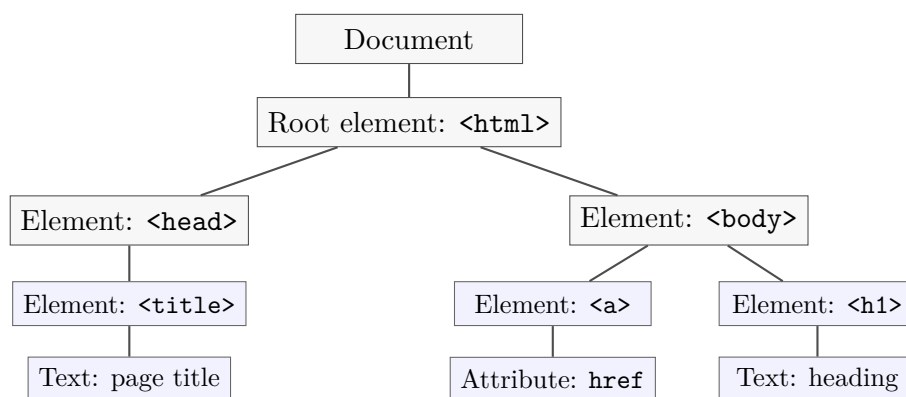


Figure 3.2: Simplified DOM tree showing how a browser represents an HTML document.

For streaming sites, the DOM is useful but incomplete. It can reveal match cards, buttons, iframes, scripts, and player containers, but it may not reveal media URLs that are generated at runtime. This is why DOM inspection is combined with screenshots and network monitoring.

3.2.2 DOM signals on streaming pages

Different page roles expose different DOM signals. Landing pages expose repeated containers. Hosting pages expose player containers and server controls. Embedded pages often expose a reduced tree with one `<video>` element or a JavaScript player root.

DOM signal	Where it appears	Why it matters
Repeated cards or rows	landing pages	indicates match listings and candidate links
<code></code>	landing and hosting pages	provides navigation targets
<code>data-*</code> attributes	landing and dynamic pages	may store event IDs, team names, or hidden URLs
<code><iframe src="..."></code>	hosting pages	points to embedded player context
<code><video></code> or <code><source></code>	hosting or embedded pages	may expose direct media source references
script text and JSON blocks	all page types	may contain player configuration or lazy-loaded data

Table 3.2: DOM signals commonly used to understand streaming-site pages.

The conclusion is that DOM analysis is a starting point, not a complete solution. It identifies page structure and interaction targets, while runtime browser evidence confirms what actually happened after the page executed.

3.3 Streaming Protocols and Embedded Players

The final evidence in this domain is usually not a normal web page. It is a media manifest, a player configuration, or a network request made by the browser while the player is active. This section explains the main protocols and player embedding patterns at a general level.

3.3.1 Common embedding patterns

Hosting pages usually embed players in one of three ways. The simplest case is a direct `<video>` element with a `src` or nested `<source>` tag. A common case is an `iframe` that loads a player from another domain. A more dynamic case is a JavaScript player library that inserts the video source after page load.

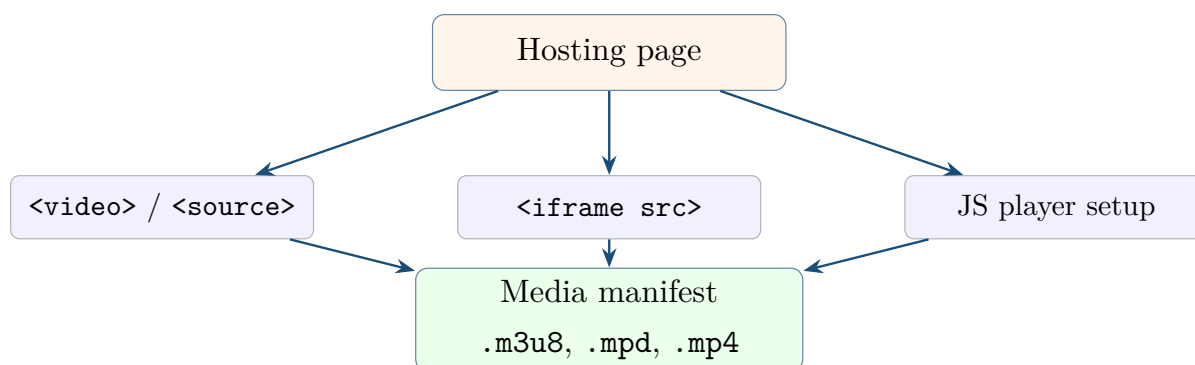


Figure 3.3: General embedding paths from a hosting page to a media manifest.

This distinction matters because each embedding type requires a different observation strategy. Direct video tags can be read from the DOM. Iframes require frame traversal. JavaScript players often require runtime execution and network monitoring.

3.3.2 HLS and MPEG-DASH

HTTP Live Streaming (HLS) is one of the most common protocols for web video streaming. Its entry point is a .m3u8 playlist that points either to variant playlists or to media segments. MPEG-DASH uses an XML .mpd manifest that describes periods, representations, and segment addressing rules. Both protocols are adaptive: the player can choose different quality levels depending on network conditions [22, 23].

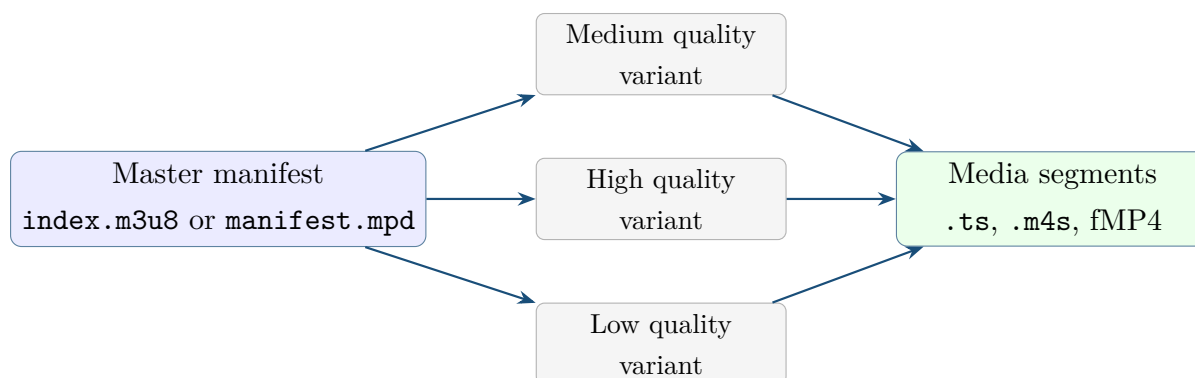


Figure 3.4: Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH.

For evidence collection, the manifest URL is usually more important than downloading media segments. It proves that the page resolved to a playable stream entry point and often contains CDN, token, and path information.

3.3.3 Tokens, cookies, referers, and provider checks

Streaming manifests are frequently protected by short-lived tokens, cookies, referer checks, or provider-side gates. A URL such as `playlist.m3u8?token=...` may work only for a few minutes and only from the browser session that generated it. CDNs and fronting providers such as Cloudflare can also enforce challenge pages, bot checks, cache rules, and origin protection.

This is why the browser session matters. A raw HTTP request may fail even when the browser can play the stream, because the browser carries the active cookies, headers, referer, and JavaScript-generated state. The technical background therefore includes both media protocols and web-provider behavior.

3.4 Browser Automation, Scraping, and Runtime Obstacles

Static scraping means downloading HTML and parsing it. That is useful for simple pages, but it is not enough for pages that depend on JavaScript, user clicks, iframes, or runtime media requests. Browser automation tools such as Puppeteer execute the page in a real Chromium browser and expose APIs for clicking, reading the DOM, taking screenshots, and listening to network activity [7, 12].

3.4.1 Puppeteer as a browser control layer

Puppeteer provides programmatic control over Chromium. In this project context, the important concept is not only “click a button” but full runtime observation:

- inspect the DOM after JavaScript has executed;
- click buttons, tabs, overlays, server selectors, and play controls;
- traverse frames and iframes;
- capture screenshots for human review;
- observe network requests through the Chrome DevTools Protocol;
- read browser-side state with `page.evaluate(...)`.

Puppeteer is therefore the bridge between a high-level extraction objective and the actual state of the web page. It lets the system test what a user would see and what the browser actually requested.

3.4.2 Ads, cookies, popups, and other annoyances

Illegal streaming websites often monetize through aggressive ad behavior. These behaviors are not just inconvenient; they change the extraction problem because the automation may click the wrong layer, follow an ad redirect, or lose the original player context.

Obstacle	Common symptom	General handling idea
Popups and new tabs	clicking play opens an ad page instead of the player	close extra targets and retry on the original page
Overlay layers	a fake play button covers the real player	detect visible overlays and click or remove them carefully
Cookie banners	controls are blocked until consent is handled	accept, dismiss, or record as blocking evidence
Ad iframes	DOM contains many unrelated frames	rank frames by player-like signals instead of following every iframe
Cloudflare-like challenges	browser sees an interstitial instead of the page	wait, retry with normal pacing, or preserve a blocked-state result

Table 3.3: Common browser-level obstacles in streaming-site extraction.

Ad blocking tools such as uBlock Origin can reduce noise by blocking known ad and tracking domains [24]. However, they cannot replace browser reasoning. If a site serves ads and player assets from the same domain, aggressive blocking can break the player itself. The general solution is layered: block obvious noise, keep popup recovery, and preserve evidence when the page remains blocked.

3.4.3 Why screenshots and network logs matter

DOM text alone does not prove that a player actually loaded. A screenshot can show that the player was visible or that a challenge blocked the page. A network log can show that the browser requested a media manifest. Together, DOM, screenshot, and network evidence give a more reliable picture than any one source by itself.

This is also why extraction tools should avoid claiming success only because a keyword or button was found. In this domain, useful evidence is the combination of page role, visible state, player behavior, and media request.

3.5 Agentic Evidence Collection Concepts

The project uses the term *agentic* to describe an extraction approach where a language-model-driven agent observes a page, chooses a tool action, checks the result, and repeats until it has enough evidence or reaches a limit. This differs from a fixed scraper because the next action depends on the current browser state.

3.5.1 Agents and ReAct-style loops

Agentic systems commonly use a loop that combines reasoning and tool use. In a ReAct-style pattern, the agent observes the state, reasons about what to do, acts through a tool, and verifies the result before continuing [1, 3].

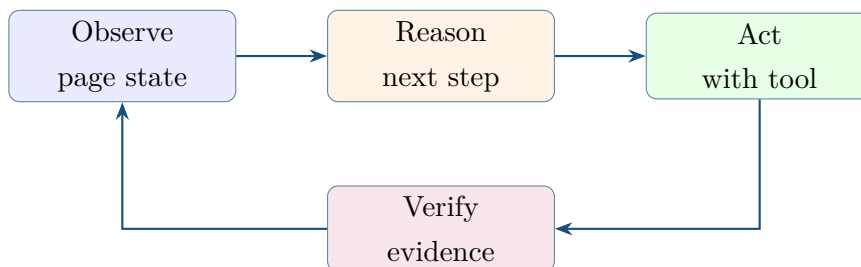


Figure 3.5: General ReAct-style loop used by browser agents.

For streaming sites, this loop is useful because the agent can adapt when a server button fails, a popup opens, an iframe appears, or a page turns out to be unrelated. The agent is not expected to know the full path in advance; it uses tools to discover it.

3.5.2 Prompt engineering, tokens, and token usage

Prompt engineering is the process of defining the agent’s role, constraints, tool usage rules, evidence requirements, and output format. In this project domain, a prompt must prevent premature conclusions. It should tell the agent to ground decisions in visible page state, DOM evidence, screenshots, and network observations.

Tokens are the units used by language models to process prompts and outputs. Long page snapshots, repeated tool results, and verbose traces increase token usage and therefore increase cost and latency. Token management is part of the technical background because browser agents can easily spend too much context on noisy HTML, ads, or repeated page states.

Practical token-control ideas include:

- summarize page observations instead of passing full raw HTML every time;
- keep only evidence-relevant DOM snippets and network URLs;
- stop early when the page role or stream evidence is clear;
- separate input tokens, output tokens, and cached tokens when estimating cost;
- use provider-side caching when stable prompt sections are reused.

LLM providers such as Gemini expose model features that affect cost, context length, streaming behavior, and caching [6]. Provider caching is especially relevant when the same system instructions, tool descriptions, or long context blocks are reused across many runs.

3.5.3 n8n and workflow automation

n8n is a workflow automation platform that lets developers connect nodes for HTTP requests, browser actions, LLM calls, data transformations, and external services through a visual flow [18]. It is useful for fast validation because a workflow can be assembled and adjusted without building a full application from the beginning.

In this report, n8n should be understood as a workflow orchestration tool. It can sequence steps, pass data between nodes, call LLM providers, and make the evidence collection logic visible during early experimentation. Later chapters explain how the validated workflow relates to the more structured engineering design.

3.5.4 LangChain, tools, and MCP servers

LangChain is a framework for building applications around language models, prompts, tools, memory, and structured chains. At a general level, it helps connect an LLM to external capabilities: a browser tool, a database lookup, a parser, a memory store, or a custom function. This is useful when the model should not only answer textually but take controlled actions.

The Model Context Protocol (MCP) is a way to expose external tools and resources to an agent through a consistent interface. In this type of project, an MCP server can expose browser tools such as inspect, click, navigate, screenshot, harvest, or media-state checks. The agent does not need to know how each tool is implemented internally; it needs a clear tool contract, input schema, and output shape.

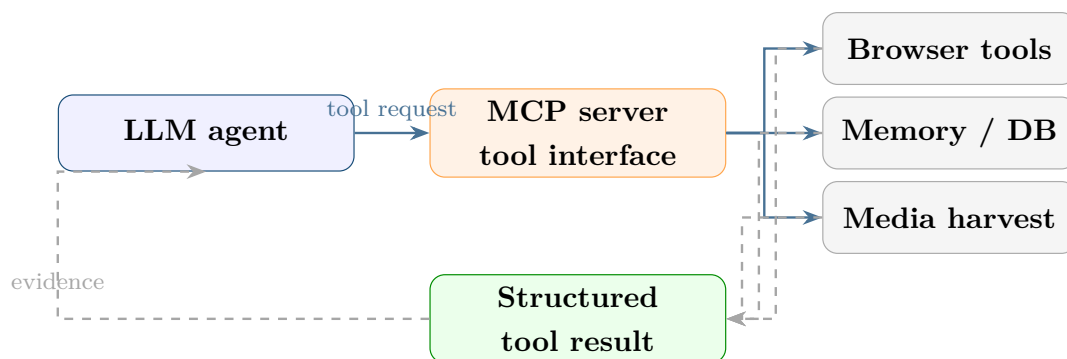


Figure 3.6: General relationship between an LLM agent, MCP tool interface, and external tools.

The important concept is separation of responsibility. The LLM decides what to try next, while tools perform bounded actions and return structured evidence. This keeps the agentic process auditable.

3.6 Chapter Summary

This chapter established the technical background for the rest of the report. It explained the page flow from landing pages to hosting pages and embedded players, the DOM structures used to inspect those pages, the streaming protocols and player embeds that carry the final evidence, and the browser-level obstacles caused by ads, cookies, popups, iframes, and provider-side checks. It also introduced the agentic concepts used later: ReAct-style loops, prompt engineering, tokens, provider caching, n8n, LangChain, and MCP tool servers.

The next chapters use these concepts more concretely. Chapter 4 translates them into architecture, and Chapter 5 explains how the implementation applies them in the actual evidence-collection system.

CHAPTER 4

System Architecture

This chapter describes the system at architecture level before Chapter 5 discusses implementation details. The objective is to keep the overview readable: diagrams show the main structure and tables carry the detailed responsibilities. This avoids diagrams that try to encode every internal connection and become hard to read.

The architecture is presented through four views: use cases, logical run flow, code/data structure, and physical runtime. Together, these views explain how an operator starts an investigation, how agents cooperate, which services support the workflow, and where the main runtime components are located.

4.1 System Overview and Use Cases

The platform is an evidence-collection workbench for illegal streaming websites. The main actor is the operator, who launches runs and reviews evidence. The engineer or administrator maintains prompts, models, datasets, browser runtime settings, and tool configuration. External services provide LLM inference, target website content, screenshot storage, and provider ownership data.

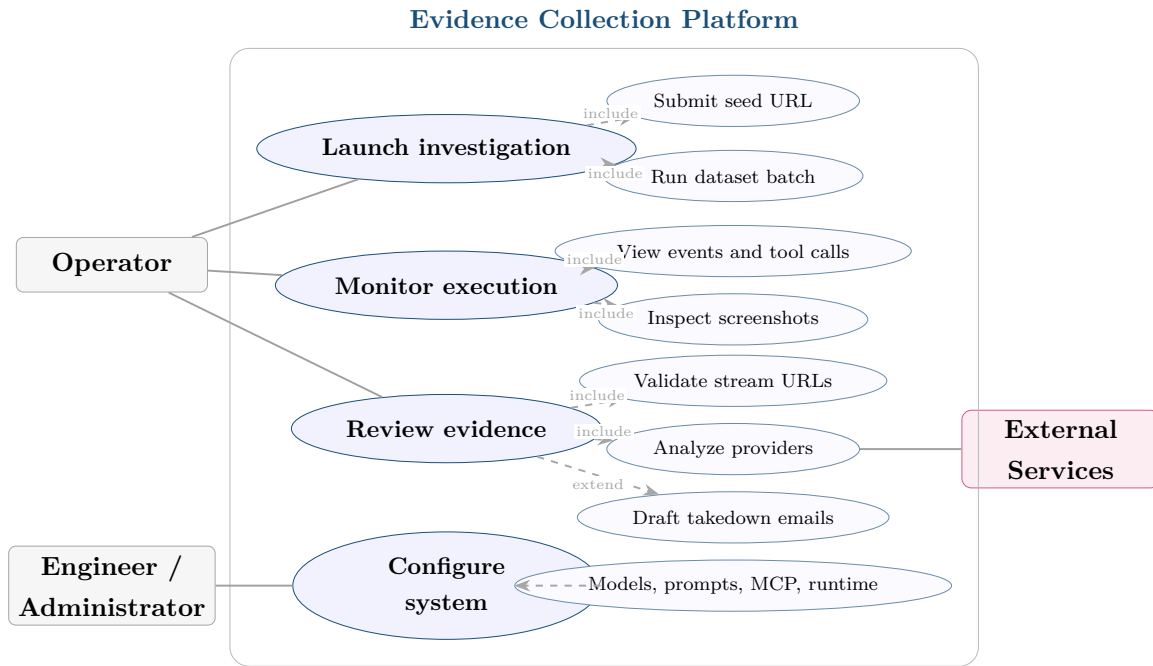


Figure 4.1: Use-case overview of the evidence-collection platform.

Use-case group	What the user does	Main system response
Launch investigation	Submit one URL or a dataset batch.	Create a run, validate runtime readiness, initialize trace and cost tracking.
Monitor execution	Follow events, tool calls, agent transitions, screenshots, and failures.	Stream structured run events and persist each important step.
Review evidence	Inspect streams, screenshots, provider ownership, and email drafts.	Assemble a reviewable evidence bundle with status and metrics.
Configure system	Update models, prompts, browser runtime, MCP tools, pricing, and datasets.	Persist settings and expose health/preflight feedback.

Table 4.1: Main use-case groups and system responsibilities.

4.2 Logical Agent Architecture

The logical workflow is intentionally staged. Classification decides the page type, the orchestrator routes work to the correct specialist, and extracted evidence is enriched before review. The most important routing rule is that landing, hosting, and embedded

pages are different page types, so they require different browser strategies.

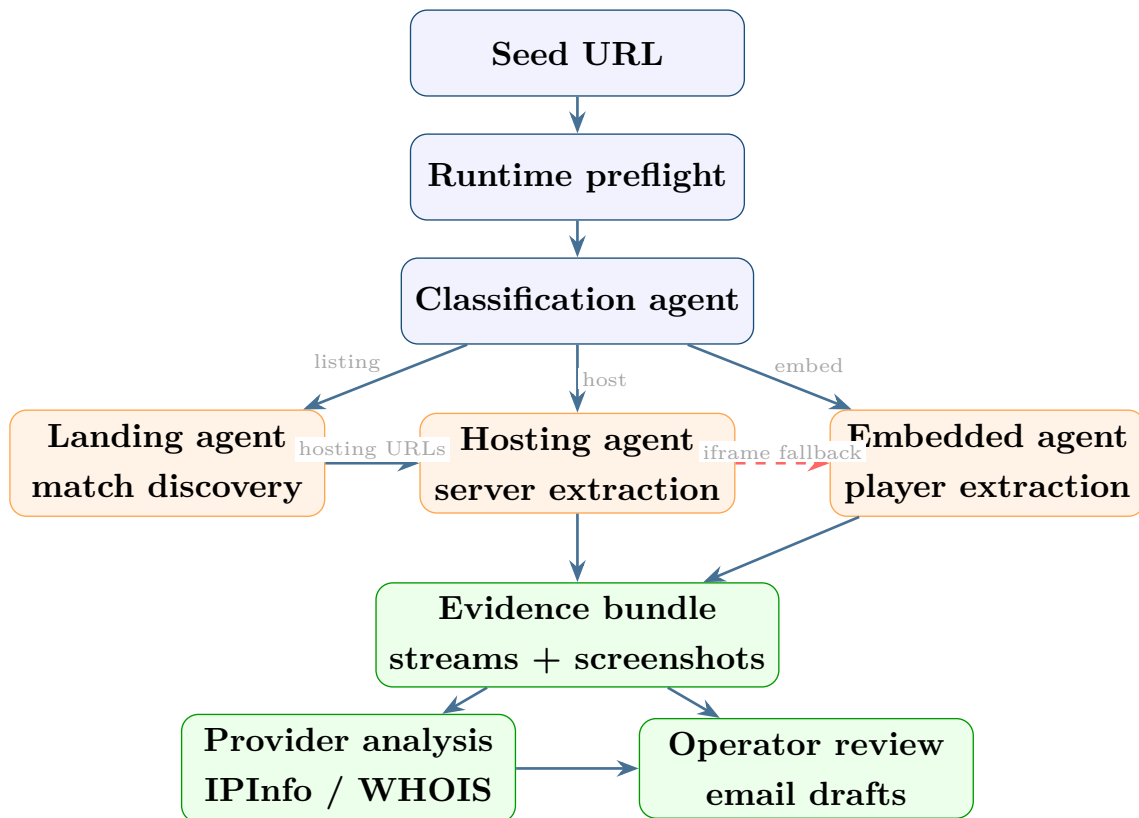


Figure 4.2: Logical workflow of an evidence-collection run.

Figure 4.3 shows the agent interconnection at a higher level. The orchestrator is the only component that routes between specialists. Shared services are shown as support blocks, not as many individual arrows, because the detailed tool allocation is covered in Chapter 5.

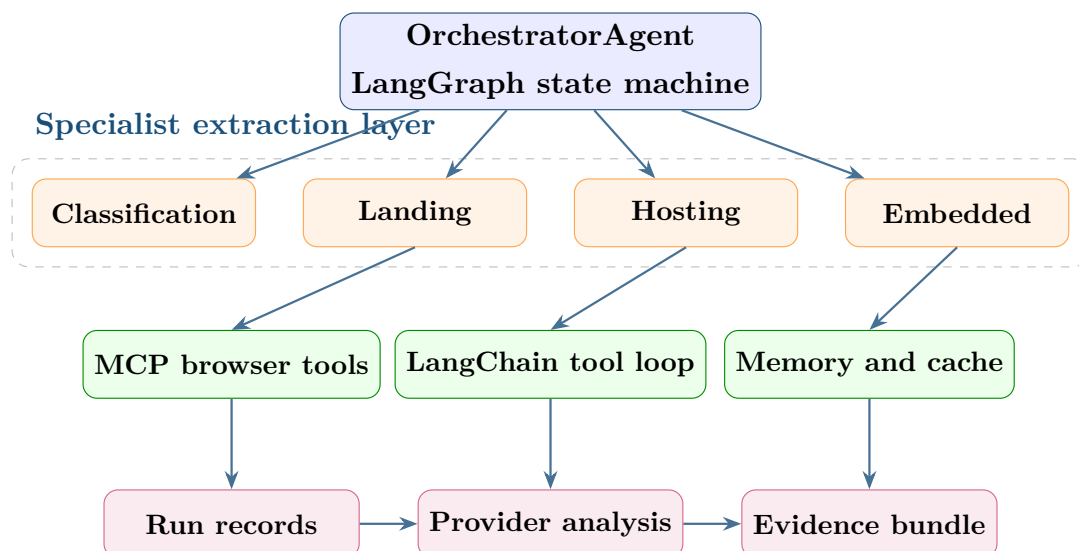


Figure 4.3: Logical interconnection of the orchestrator, specialists, and shared support services.

4.2.1 Run Sequence

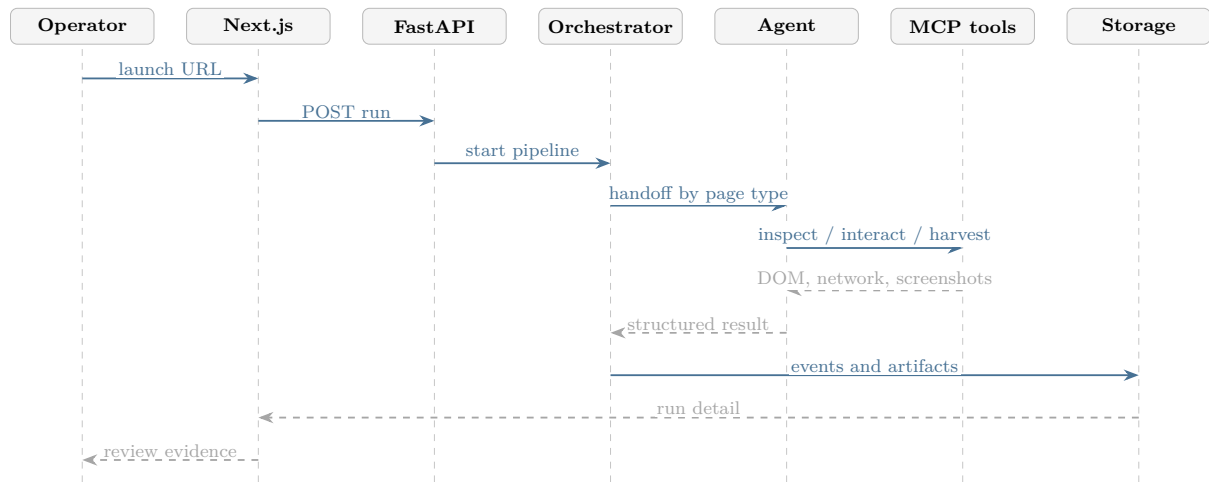


Figure 4.4: Sequence diagram for a standard investigation run.

4.3 Static Code and Data Model View

The static view is intentionally compact. Instead of drawing every class relationship, Figure 4.5 shows the main code families and the dependency direction between them. The data records are grouped into observability, evidence, configuration, and evaluation records.

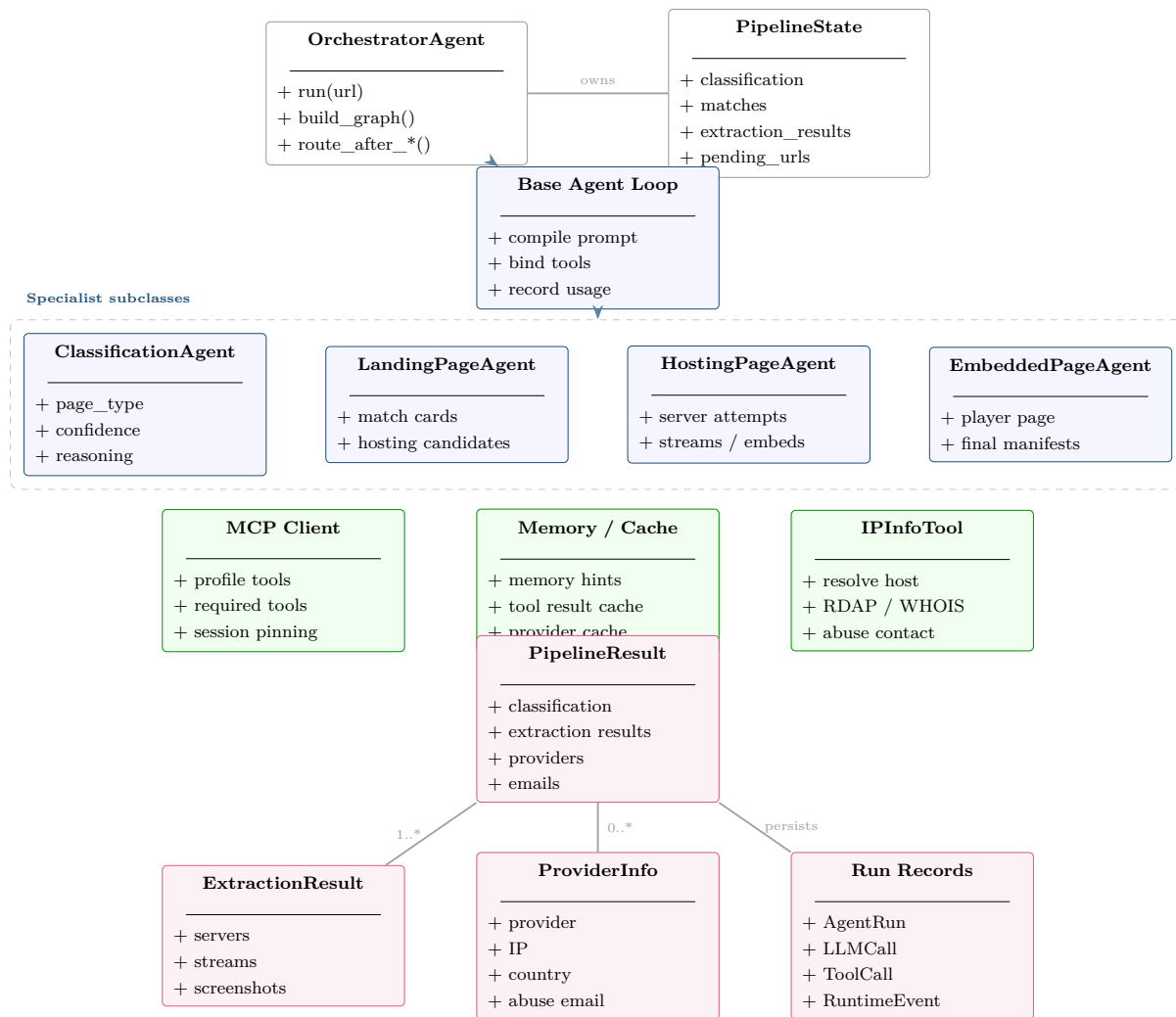


Figure 4.5: UML-style class diagram for the agent runtime and stored outputs.

Architectural element	Responsibility	Main output
Orchestrator	Owns run state, routing, handoff context, and stop conditions.	Pipeline result and route decisions.
Specialist agents	Execute page-type-specific extraction strategies.	Classification, match candidates, server attempts, stream evidence.
MCP browser tools	Expose controlled browser actions to each agent profile.	DOM snapshots, screenshots, iframe trees, media state, network harvests.
Storage layer	Keeps the execution reviewable after the browser run ends.	Events, tool calls, LLM calls, screenshots, streams, providers, costs.

Table 4.2: Main code responsibilities and outputs.

4.4 Physical Runtime and Supporting Services

The physical view shows runtime placement and service boundaries. The diagram is kept linear on purpose: the operator talks to the frontend, the frontend talks to the API, the API launches agents, agents use MCP tools, and the browser interacts with the target site. Storage, cache, model, and provider services are shown as supporting services.

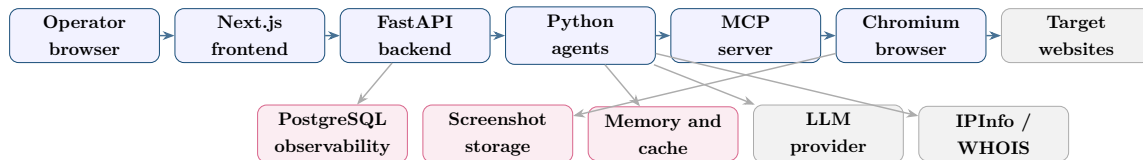


Figure 4.6: Component architecture of the system runtime.

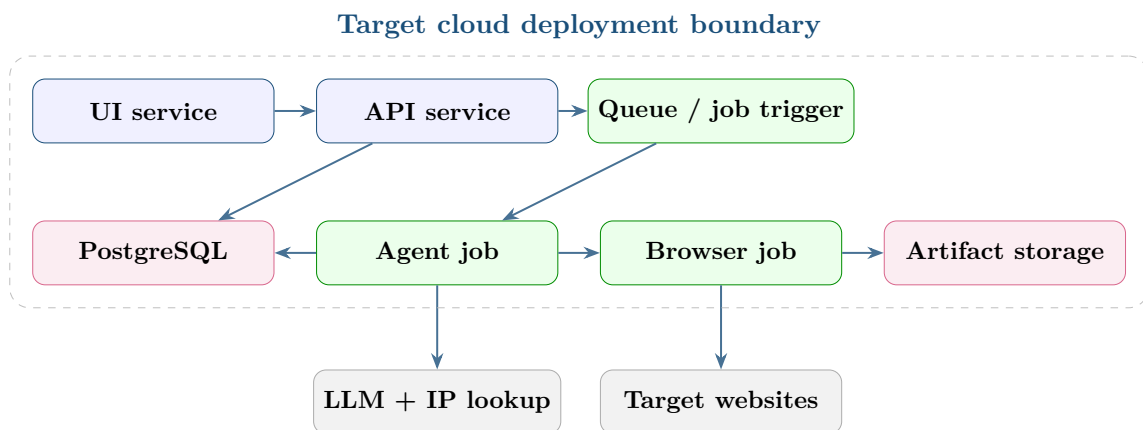


Figure 4.7: Physical runtime and hardware-oriented deployment view.

4.4.1 Validation and Repair Loop

Browser automation cannot be a static scrape in this domain. Popups, ad tabs, cookie banners, Cloudflare-style interstitials, unstable selectors, and nested iframes can change the state after each click. The runtime therefore uses a simple observe-act-validate loop.

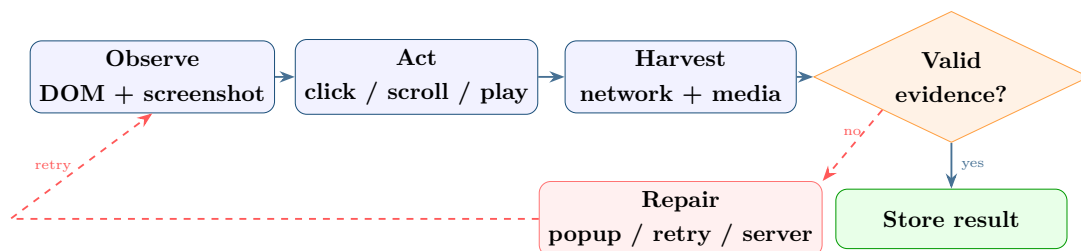


Figure 4.8: Browser validation and repair loop used by specialist agents.

4.5 Memory, Caching, and Deployment Summary

Memory and caching are support mechanisms, not separate product features. They exist to reduce repeated work, preserve useful domain knowledge, and make cost visible.

Layer	Scope	Purpose
Short-term memory	One run and one agent session.	Keep visited URLs, recent tool observations, and recovery context available during the run.
Long-term memory	Cross-run domain knowledge.	Reuse known selectors, layout patterns, iframe behavior, provider domains, and successful strategies.
Tool result cache	Repeated browser observations.	Avoid repeating equivalent inspect, context, screenshot, or media-state calls.
Provider cache	Repeated static prompt context.	Reduce repeated instruction tokens when provider-side cache is available.
Usage records	Run review and evaluation.	Track input, output, cached tokens, cost, and model-call metadata.

Table 4.3: Memory and caching layers used by the runtime.

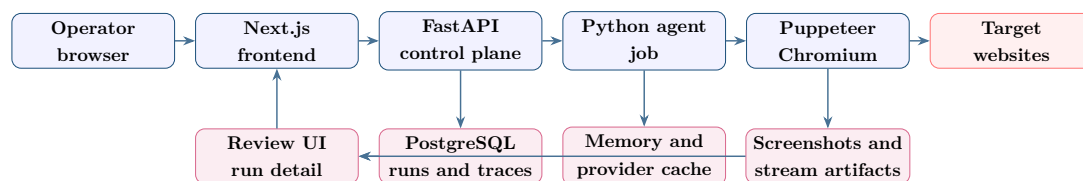


Figure 4.9: Target deployment architecture with separated UI/API services and browser-heavy jobs.

4.6 Chapter Summary

This chapter defined the system at architecture level. The key idea is a staged, reviewable evidence-collection pipeline: the operator starts and reviews runs, the API controls execution, the orchestrator routes across page-type specialists, MCP browser tools gather evidence, and storage preserves the result for later inspection. Chapter 5 now moves from this architecture to the concrete implementation.

CHAPTER 5

Implementation

The implementation progressed in two major parts. The first part was an n8n workflow used as a fast exploration and validation environment. The second part was a Python reference solution built around FastAPI, Next.js, MCP browser-tool containers, specialist agents, model/runtime management, and PostgreSQL observability.

This split matters because the two systems did not have the same purpose. The n8n workflow was useful for testing whether agentic browsing could handle the domain at all. The Python implementation was built after those lessons, with cleaner engineering boundaries, persistent traces, configurable runtime settings, and a better foundation for evaluation.

5.1 Part 1: n8n Workflow Prototype

5.1.1 Purpose of the n8n phase

The n8n implementation acted as the project playground. It made it possible to test the core question quickly: can an LLM-driven browser agent inspect illegal streaming websites, navigate through landing and hosting pages, activate players, and recover useful evidence? At this stage, the priority was not a clean software architecture. The priority was proving feasibility on real, unstable websites.

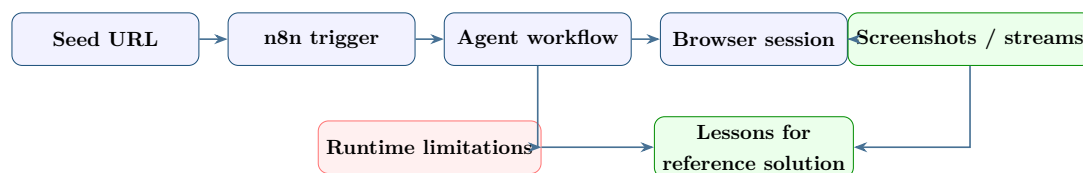


Figure 5.1: Role of the n8n workflow as a feasibility playground.

The workflow helped test agent behavior before investing in a full backend, database, and operator console. It also exposed which parts of the task were hard: browser state continuity, popup handling, stream activation, memory, runtime length, and evidence validation.

5.1.2 n8n agents and screenshot placeholders

The n8n workflow used agent-like nodes to test the same page-type responsibilities that later became specialist agents. The screenshots below are intentionally left as placeholders so the final report can include the real workflow captures.

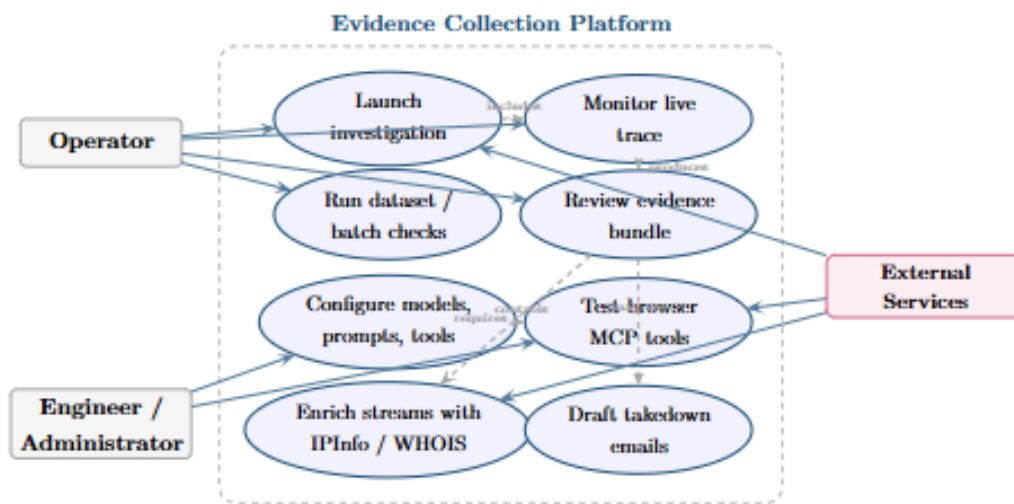


Figure 4.1: Use-case overview of the evidence-collection platform.

Screenshot Placeholder n8n classification agent workflow node and output	Screenshot Placeholder n8n landing agent match discovery path
Screenshot Placeholder n8n hosting agent server and player activation	Screenshot Placeholder n8n embedded agent iframe/player extraction

Figure 5.2: Full n8n workflow and screenshot placeholders for the agent workflows.

n8n agent role	What it tested	Lesson learned
Classification	Whether a page could be categorized as landing, hosting, embedded, or other from browser evidence.	Routing must be explicit and fail-closed; wrong classification wastes the whole run.
Landing	Whether listing pages could expose match cards, tabs, and hosting links.	Generic DOM inspection was not enough; landing pages needed page-type-specific extraction.
Hosting	Whether server buttons and play controls could be activated.	Stream evidence appears only after interaction; screenshot and media-state validation are necessary.
Embedded	Whether iframe and player pages could reveal final manifests.	Nested frames require a dedicated extraction path and should not be mixed with landing-page exploration.

Table 5.1: n8n agent roles and lessons carried into the reference implementation.

5.1.3 Limitations of the n8n implementation

The n8n workflow was valuable, but it reached clear limits when the task became larger than a single experimental run. These limits explain why the second implementation was necessary.

Limitation	What happened in practice	Consequence
No real evaluation layer	Runs could be inspected manually, but there was no structured evaluation harness for repeatable scoring.	Prompt changes were hard to compare objectively.
Weak memory model	The workflow did not provide a durable, domain-aware memory layer for reusable site knowledge.	Agents repeated failed actions and could not reliably reuse successful selectors or strategies.
Long runtimes	Large pages and multi-server hosting pages created long blocking executions.	Debugging became slow and failures were expensive to reproduce.
Sequential execution	The workflow behaved mostly as a sequential chain rather than parallel specialist agents.	Runs were slower than necessary, especially when multiple hosting or embedded candidates existed.
Limited development environment	n8n is a workflow editor, not a normal source controlled development environment.	Workflows had to be exported as JSON files to preserve progress and compare versions.
Fragile Puppeteer integration	Puppeteer code inside n8n nodes was brittle and tools frequently broke when browser state changed.	Browser automation needed a dedicated tool service with stable sessions.
Isolated code execution	Code executions were isolated, so browser instances could be lost between steps. A <code>puppeteer.connect</code> workaround was needed to reconnect to a running browser.	Browser continuity became a workaround instead of a clean runtime contract.
High memory usage	Long browser sessions, page artifacts, and workflow state created large memory pressure.	The prototype was not comfortable for long or batch-oriented extraction runs.
Hard failure analysis	Errors were visible, but not stored as normalized events, LLM calls, tool calls, screenshots, and costs.	Post-mortem review required manual reconstruction.

Table 5.2: Main limitations observed during the n8n prototype phase.

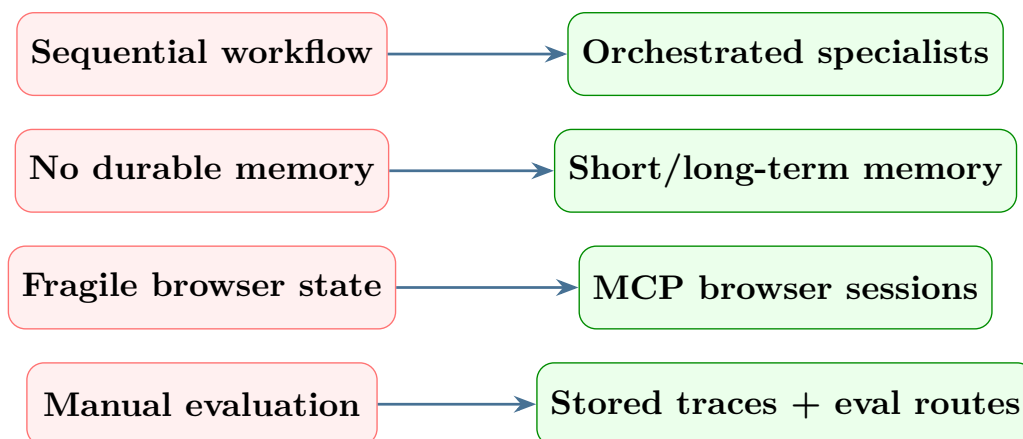


Figure 5.3: How n8n limitations shaped the reference implementation.

5.2 Part 2: Implemented Reference Solution

The reference solution keeps the useful extraction logic from the n8n phase but moves it into explicit software components. It is organized into five main implementation areas: FastAPI, Next.js, MCP browser containers, agents and model runtime, and PostgreSQL.

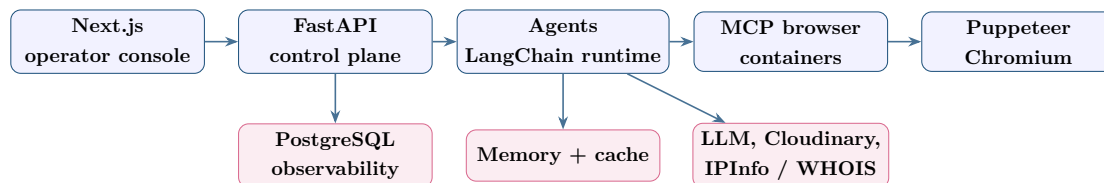


Figure 5.4: Five implementation areas of the reference solution.

5.2.1 FastAPI Backend

The FastAPI backend is the control plane of the reference solution. It starts runs, checks runtime readiness, streams or returns run state, exposes settings, stores observability records, and gives the frontend a stable API surface.

Backend area	Responsibility	Why it matters
Runtime health	Check browser, MCP, settings, and launch readiness.	Failed preflight is clearer than a silently broken agent run.
Run execution	Start single URL runs and dataset-backed runs.	Keeps workflow launch separate from UI rendering.
Run inspection	Return trace, screenshots, streams, provider data, model usage, and errors.	Makes each run auditable after execution.
Configuration	Persist model, provider, pricing, browser, and tool settings.	Allows controlled experimentation without editing code every time.
Tool playground	Let tools be tested without a full agent run.	Isolates browser/tool failures from prompt failures.
Evaluation routes	Run or review batches and reliability checks.	Provides the foundation that the n8n prototype lacked.

Table 5.3: FastAPI backend responsibilities.

The backend also uses lightweight caching for frequently polled read endpoints. This reduces repeated database work when the frontend refreshes dashboards or run summaries.

5.2.2 Next.js Operator Console

The Next.js frontend is not just a dashboard. It is the operator console for the reference implementation: launching runs, following execution, reviewing evidence, and changing runtime configuration.

Frontend surface	What it shows or controls	Main backend dependency
Run launcher	Seed URL, batch inputs, runtime readiness, launch status.	Workflow and health endpoints.
Run detail	Agent activity, events, decisions, screenshots, streams, provider analysis, tool calls, model usage.	Run inspection endpoints and stored traces.
Datasets and batches	Website dataset, batch launch, per-site progress, history.	Dataset and batch repositories.
Settings	Provider/model config, caching, browser runtime, pricing, MCP tools.	Configuration endpoints.
Tool playground	Direct tool tests for navigation, inspection, screenshots, and harvest.	MCP tool-test endpoints.

Table 5.4: Main Next.js operator workflows.

The design goal was operational clarity. A failed launch should show why it failed. A completed run should show what the agent actually did, which tools it called, what evidence it found, and where the model or browser runtime failed.

5.2.3 MCP Browser Containers and Puppeteer Tools

The browser tooling was extracted from the n8n code-node style into dedicated MCP browser services. Each agent profile receives a filtered tool surface through an MCP endpoint. This reduces tool-selection noise and keeps browser session ownership outside the LLM prompt.

Browser engine	Strengths observed	Project decision
Playwright	Strong context isolation, good frame and browser-context APIs, useful for comparing behavior and validating browser policies.	Tested and kept as a useful reference path, but not the primary runtime.
Puppeteer	Direct Chrome DevTools Protocol fit, simpler integration with the existing tool registry, stable enough for the profiled MCP tool server.	Chosen as the main browser engine for the implemented agent prompts and tool runtime.

Table 5.5: Playwright and Puppeteer comparison for the browser tool layer.

Tool group	Purpose
Navigation tools	Open URLs, go back, wait for page state, and recover from navigation drift.
Inspection tools	Read page context, landing cards, hosting server controls, embedded player state, DOM elements, and frame trees.
Interaction tools	Click by selector, text, XPath, checkbox/radio/select, coordinates, play media, type input, scroll, and swipe.
Evidence tools	Capture screenshots, media state, stream requests, player signals, and harvested m3u8/mpd/mp4 candidates.
Memory tools	Read or update reusable site hints from the memory layer.

Table 5.6: Puppeteer tool groups exposed through MCP.

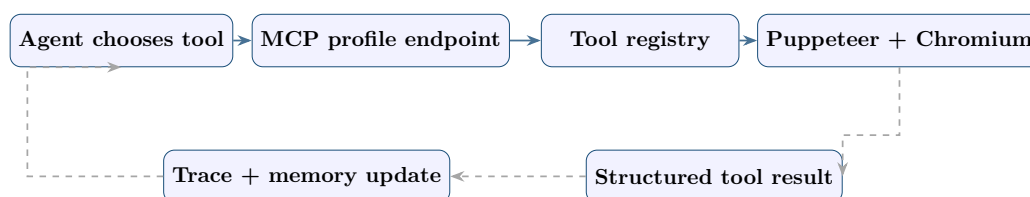


Figure 5.5: Browser tool execution loop through MCP and Puppeteer.

5.2.4 Agents, Models, Memory, and Prompts

The reference solution uses specialist agents rather than one general browser agent. Each specialist receives a role-specific prompt, a profile-specific tool set, and a bounded execution budget.

Agent	Objective	Main tools used
Classification	Decide whether the seed is landing, hosting, embedded, or other.	Generic inspect, screenshot, frame tree, memory lookup.
Landing	Discover match cards, listing tabs, pagination, and hosting candidates.	Landing inspect, element queries, click/scroll, screenshot, memory tools.
Hosting	Activate server controls, verify player state, and extract streams or embedded fallbacks.	Hosting inspect, click/play tools, media state, frame tree, harvest, screenshot.
Embedded	Work only on embedded/player URLs and harvest final stream evidence.	Embedded inspect, play/media tools, harvest, screenshot, frame diagnostics.

Table 5.7: Specialist agents, objectives, and tool surfaces.

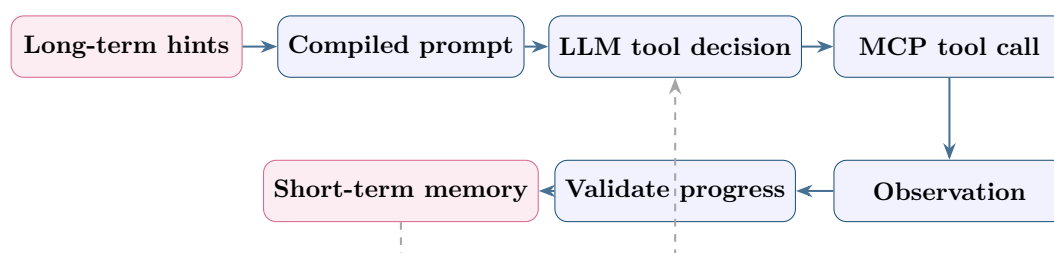


Figure 5.6: Agent reasoning loop with memory and tool feedback.

LangChain is used as the tool-binding and message-loop layer. The implementation does not let the model freely access every browser capability. Instead, the MCP client loads only the tool profile for the current agent. The shared loop records tool calls, model calls, token usage, cache behavior, and stop reasons.

Model behavior, token usage, and rate limits

The model layer had to be treated as an operational dependency, not only as a prompting detail. Long browser pages, repeated DOM observations, screenshots, tool traces, and failed retries can grow the context quickly. The implementation therefore tracks context windows, input tokens, output tokens, cached tokens, cache writes, and estimated cost.

Gemini rate limits are evaluated across RPM, TPM, and RPD. Google’s current documentation defines these as requests per minute, input tokens per minute, and requests per day, and states that limits depend on model and usage tier and can change in AI Studio.¹ The implementation therefore treats rate limits as configuration and observability

¹<https://ai.google.dev/gemini-api/docs/rate-limits> Last checked on 2026-05-18.

concerns rather than hard-coded constants.

Model concern	Implementation handling	Why it matters
Context window	Store and display model context-window metadata where available.	Prevents long runs from silently approaching context exhaustion.
Token usage	Track input, output, cached input, and cache-write tokens.	Makes cost and prompt growth visible per run.
Tool calling	Bind only profile-specific tools through LangChain and MCP.	Reduces incorrect tool choice and keeps browser actions controlled.
Rate limits	Treat RPM, TPM, and RPD as provider limits to monitor and respect.	Avoids confusing quota failures with browser or prompt failures.
Retries	Apply bounded LLM retry attempts and delays per agent profile.	Recovers transient provider errors without infinite loops.
Caching	Use prompt/provider cache and tool-result cache where safe.	Reduces repeated tokens and repeated browser observations.

Table 5.8: Model-runtime concerns handled by the implementation.

Prompt engineering and stopping behavior

The prompts evolved from broad goals into operational procedures. Each prompt defines the objective, the evidence required, which page ownership rules must be respected, and when the agent should stop. The strongest prompt pattern was an evidence-based loop: observe, state the current hypothesis, choose one action, verify the result, then either continue or return a structured failure.

Prompt design rule	Implementation effect
Explicit page ownership	Hosting agents stay on hosting pages; embedded agents stay on embedded/player URLs.
Evidence before harvest	Agents confirm player state or screenshot state before claiming stream extraction.
Bounded tool budget	Long runs stop with a clear partial result instead of looping indefinitely.
Structured output contract	Results can be stored, evaluated, and rendered in the UI without manual parsing.
Memory hints as soft guidance	Prior site knowledge helps, but does not replace current evidence.

Table 5.9: Prompt engineering rules used by the specialist agents.

5.2.5 PostgreSQL Database and Observability

The PostgreSQL database is used mainly for observability, review, evaluation, and configuration. It is not the product database of the external organization. Its purpose in this project is to preserve what happened during a run so that failures and successes can be inspected later.

Record group	Tables / records	Status in the project
Core run trace	Pipeline runs, agent runs, runtime events, LLM calls, tool calls.	Central to reviewing and debugging runs.
Evidence	Run streams, screenshots, provider analyses, takedown emails.	Central to the evidence bundle.
Model and prompt metadata	Prompt versions, prompt compilations, model usage, pricing configs.	Important for cost, prompt iteration, and reproducibility.
Memory	Memory entries and memory hints used.	Used to connect cross-run knowledge with current agent decisions.
Datasets and batches	Dataset sites, dataset batches, dataset site runs.	Useful for evaluation/batch execution, but not always used in every run.
Operational support	Run decisions, run tasks, playground calls, snapshots.	Support/diagnostic records; some remain secondary rather than core report outputs.

Table 5.10: PostgreSQL record groups and their role in the implementation.

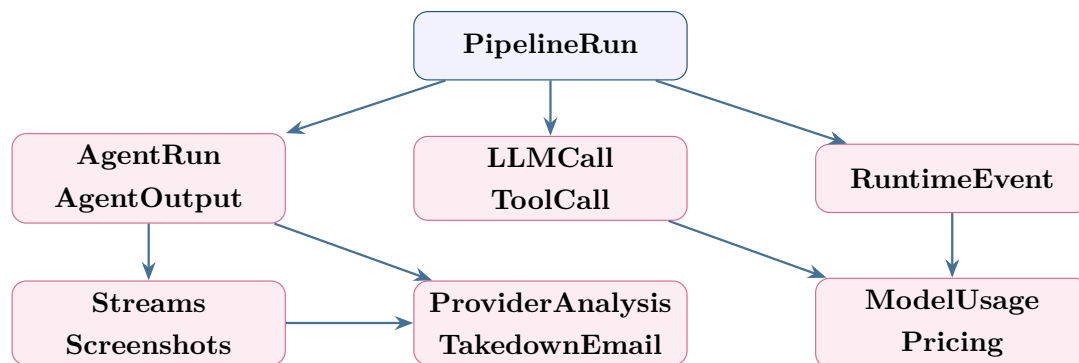


Figure 5.7: Simplified database relationship map for run review.

5.3 Implementation Summary

The implementation moved from a useful but limited n8n prototype to a structured reference solution. The n8n phase validated the feasibility of agentic browser extraction and exposed the main runtime problems. The Python solution addressed those problems with explicit backend routes, a reviewable frontend, profiled MCP browser tools, specialist agents, memory and caching, model-usage tracking, and PostgreSQL observability.

The result is not only an extraction script. It is a reviewable agent platform where failures can be diagnosed, prompts can be evaluated, tool behavior can be tested independently, and evidence can be inspected after the browser run ends.

CHAPTER 6

Evaluation and Testing

This chapter evaluates the implemented system from three angles. First, it explains the tests used to protect the implementation from regressions. Second, it defines the KPI formulas used by the dashboard and by the manual evaluation. Third, it compares prompt, model, tool, cache, and cost behavior across the main evidence-collection use cases.

The evaluation is intentionally split into dashboard-observed values and manual reference-set values. The dashboard values come from the local observability database used by the operator console. At the time of writing, that database contains a small engineering snapshot, not a large production benchmark. Manual reference-set values are therefore used to evaluate the browser-agent behavior more directly.

6.1 Evaluation Sources

Evaluation used four evidence sources:

- automated regression tests for routing, prompt contracts, JSON handling, dataset status, and tool behavior;
- dashboard metrics persisted in the local run database;
- manual review of browser traces, screenshots, streams, and final outputs;
- prompt/tool comparison runs over representative page patterns.

Test layer	What it checks	Why it matters
Routing tests	Orchestrator handoff after classification, unknown-page fail-closed behavior, hosting-to-embedded fallback.	Prevents the system from sending a page to the wrong specialist or continuing after an invalid classification.
Prompt contract tests	Required output fields, stopping rules, evidence fields, and agent-specific constraints.	Prevents prompt edits from silently removing required evidence or schema structure.
Tool contract tests	Broad DOM inspection shape, scoped element detail, browser tool responses, and media-harvest fields.	Keeps the LLM tool interface stable as browser tools evolve.
Dataset and run-status tests	Batch row status, run list/detail status, cancellation, and cost fields.	Ensures the dashboard reflects what actually happened in a run.
Manual trace review	Screenshots, runtime events, stream URLs, iframe paths, and provider evidence.	Confirms behavior that cannot be fully represented by static fixtures, such as popups or tokenized playback.

Table 6.1: Evaluation sources used to test the reference implementation.

6.2 Dashboard Snapshot

The dashboard is built from persisted run records. The current local snapshot contains 9 pipeline runs. It is useful for explaining the KPI calculations and the operator view, but it is too small and too test-oriented to be treated as a final production benchmark.

Dashboard field	Calculation source	Current value
Total runs	all rows in <code>pipeline_runs</code>	9
Successful runs	<code>final_status = success</code>	2
Failed runs	<code>final_status = failed</code>	4
Cancelled runs	<code>final_status = cancelled</code>	3
Partial runs	<code>final_status = partial</code>	0
Streams extracted	sum of <code>stream_count</code>	1
Provider analyses	sum of <code>provider_analysis_count</code>	0
Takedown emails	sum of <code>email_count</code>	0
Average duration	average <code>duration_seconds</code>	1.01 s
Persisted LLM calls	sum of <code>total_llm_calls</code>	0
Persisted tool calls	sum of <code>total_tool_calls</code>	0
Persisted estimated cost	sum of <code>estimated_total_cost_usd</code>	\$0.000000

Table 6.2: Dashboard snapshot from the local observability database.

The zero values for LLM calls, tool calls, and cost do not mean that these concepts are irrelevant. They mean that the current local snapshot mostly contains short synthetic or interrupted runs where model/tool telemetry was not persisted. For this reason, the dashboard snapshot is used for formulas and observability behavior, while the manual reference set is used for extraction-quality evaluation.

6.3 KPI Definitions and Formulas

The dashboard and manual evaluation use deterministic formulas. Let N be the number of evaluated runs, N_t the number of terminal runs, S successes, P partial successes, F failures, and C cancellations.

Metric	Formula and interpretation
Terminal runs	$N_t = S + P + F + C$. Running rows are excluded from final-quality rates.
Success rate	S/N_t . Measures completed runs that reached a successful final state. In the dashboard snapshot: $2/9 = 22.2\%$.
Failure rate	F/N_t . Measures terminal runs that failed. In the dashboard snapshot: $4/9 = 44.4\%$.
Cancellation rate	C/N_t . Measures runs stopped by operator or runtime control. In the dashboard snapshot: $3/9 = 33.3\%$.
Stream evidence rate	number of terminal runs with <code>stream_count</code> > 0 divided by N_t . In the dashboard snapshot, this is $1/9 = 11.1\%$.
Page classification accuracy	correctly classified pages divided by manually verified pages. This is evaluated by comparing the selected specialist path with reviewer labels.
Navigation recall	valid hosting candidates found divided by manually verified hosting candidates available on the landing page.
Player activation rate	player activation successes divided by player activation attempts. A success requires visible state change, media state change, or stream evidence.
Stream URL extraction rate	runs with valid <code>m3u8</code> , <code>mpd</code> , <code>mp4</code> , or equivalent embedded evidence divided by extraction attempts.
Tool success rate	successful tool calls divided by observed tool calls. Tool rows or runtime events can be used as the source.
Cache hit ratio	cached input tokens divided by total input tokens: $cached_input_tokens / (cached_input_tokens + new_input_tokens)$.
Cost per accepted result	total estimated cost divided by accepted evidence bundles. This is only meaningful when LLM cost telemetry is persisted.

Table 6.3: KPI formulas used by the dashboard and the manual evaluation.

6.4 Cost Estimation Formula

Cost is estimated from token usage and model pricing. The implementation records separate cost components so that cached tokens are not treated like normal input tokens. For a model with prices expressed per million tokens, the estimated cost is:

$$\begin{aligned}
Cost = & \frac{new_input_tokens}{1,000,000} \times input_price \\
& + \frac{cached_input_tokens}{1,000,000} \times cached_input_price \\
& + \frac{cache_write_tokens}{1,000,000} \times cache_write_price \\
& + \frac{output_tokens}{1,000,000} \times output_price
\end{aligned}$$

The dashboard exposes the same split as input cost, cached-input cost, cache-write cost, output cost, and total cost. When provider-reported cost is available, it can be used directly. Otherwise, the system resolves model pricing from the configured provider pricing catalog and calculates the estimate from recorded usage metadata.

Cost component	Meaning
New input cost	Cost for prompt/context tokens that are not served from cache.
Cached input cost	Lower-cost reused input tokens when provider-side caching is available.
Cache-write cost	Cost for creating or refreshing an explicit cache entry, when the provider exposes that pricing dimension.
Output cost	Cost for generated tokens. This usually has a higher price than input tokens.
Total estimated cost	Sum of all available components.

Table 6.4: Cost fields used by the dashboard and persisted model telemetry.

6.5 Manual Evaluation Protocol

Manual evaluation was required because the hardest behaviors are visible only during a real browser session: popup traps, iframe delegation, delayed stream generation, Cloudflare-like interstitials, redirects, and player state transitions.

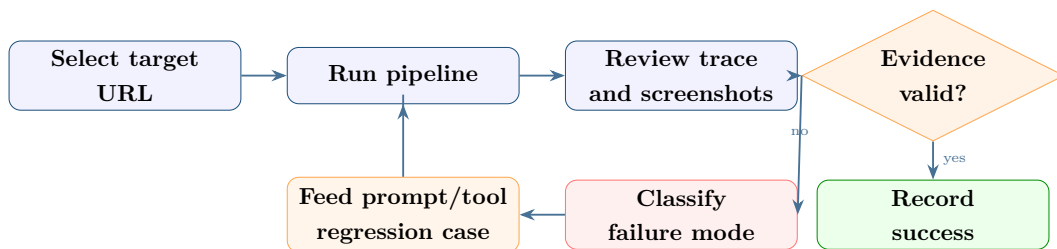


Figure 6.1: Manual evaluation loop used for browser-agent behavior.

6.6 Evaluation by Use Case and Agent

The main objective was not only to count whether the final run succeeded. The evaluation also checked whether each specialist agent completed its own responsibility.

Agent / use case	Primary KPI	Acceptance rule
Classification	Page classification accuracy	Selected page type matches manual label: landing, hosting, embedded, or other.
Landing	Navigation recall and host-candidate precision	Agent discovers valid match cards, tabs, pagination, and host-page URLs without returning unrelated links.
Hosting	Player activation, server switching, popup handling, stream URL extraction	Agent activates the player, handles overlays/popups, tries server buttons, and either extracts streams or returns embedded handoff evidence.
Embedded	Stream URL extraction, iframe traversal, screenshot reasoning	Agent focuses on the player URL, follows frame hierarchy, activates playback, and captures stream or failure evidence.
Provider analysis	Metadata extraction and provider lookup completeness	Extracted stream/provider host can be connected to IPInfo, RDAP, WHOIS, or abuse-contact evidence when available.
Operator review	Evidence completeness	Result includes status, trace, screenshot or artifact support, stream URL evidence when successful, and clear failure mode when not.

Table 6.5: Per-agent evaluation objectives and acceptance rules.

6.7 Prompt Regression Evaluation

Prompt evaluation focused on regressions: after changing a prompt, the system should not lose behaviors that were already working. The reference set grouped pages by behav-

ior: landing navigation, hosting-player activation, embedded-player extraction, redirects, popup traps, and tokenized streams.

Version	Change tested	Regression focus	Avg. tool calls
v1	Intent-based prompt	Early stop, no tab navigation, premature harvest	3.1
v2	ReAct loop and tab-check instruction	Navigation recall and screenshot-grounded decisions	6.2
v3	Popup handler and JavaScript click fallback	Popup handling and server activation after ad traps	7.8
v4	Mandatory media-state check before harvest	Player activation and premature harvest prevention	8.4
v5	Tool budget cap and early success exit	Cost reduction without losing recovery	7.1

Table 6.6: Prompt regression history and observed behavioral focus.

The main regression lesson is that lower tool-call count is not automatically better. The v1 prompt was cheaper because it stopped too early. The ReAct prompt increased tool calls because it actually inspected tabs, verified player state, and handled blockers. The useful optimization came later in v5, where early success exit reduced wasted calls without removing necessary verification.

6.8 Manual Reference-Set Results

The following values summarize the manual reference-set results used during prompt and tool iteration. They should be read as engineering evaluation values from a controlled reference set, not as population-level accuracy claims.

Metric	Intent v1	ReAct v2	Current v3/v5
Page-type routing accuracy	91%	93%	95%
Host-candidate recall	40%	85%	89%
Player activation success	0%	70%	90%
Popup-interference recovery	20%	55%	78%
Tokenized-stream recovery	25%	40%	85%
Manifest recovery on AlbaPlayer case	0%	70%	90%
Manifest recovery on tokenized Clappr case	—	40%	85%

Table 6.7: Manual reference-set results by prompt generation.

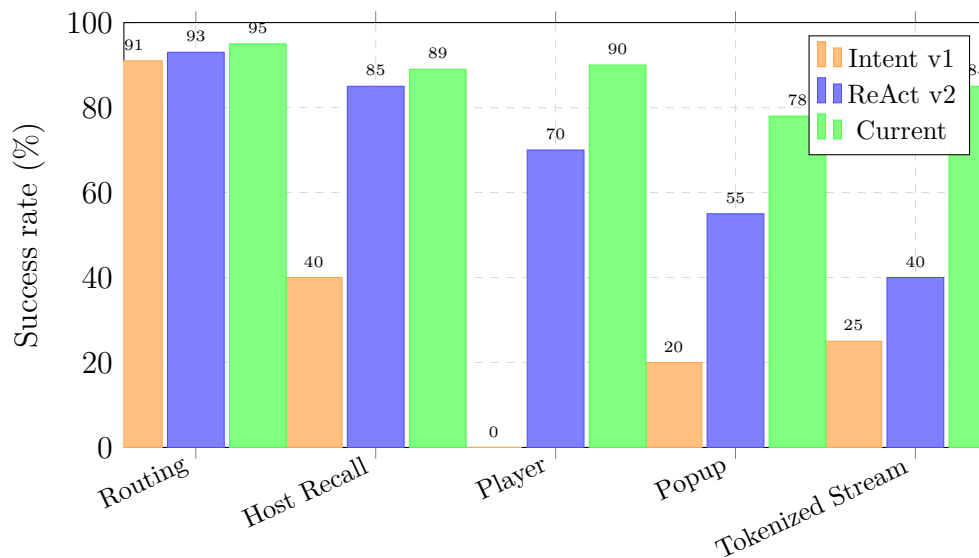


Figure 6.2: Manual reference-set improvement across the main browser-agent KPIs.

6.9 Tool Evaluation

Tool evaluation inspected both tool-call order and tool-call suitability. The most important errors were not syntax errors. They were behavioral errors: using a generic inspect

tool where a page-specific tool was needed, harvesting before playback, using screenshots as a wait loop, or navigating when the task was only a JavaScript tab click.

Observed misuse	Regression risk	Fix applied
harvest() before play click	False failure because no runtime stream has been generated yet.	Added activation guard and mandatory media-state check.
Generic inspect on host pages	Missing server controls and player-specific context.	Added page-profiled inspect tools and prompt rules.
Repeated screenshots as wait strategy	High token/cost pressure without new evidence.	Added wait parameters and screenshot checkpoints.
navigate() for same-page tabs	Reloads or loses state instead of activating dynamic content.	Clarified interact versus navigate .
Ignoring popup/new-tab context	Agent reasons over ad page instead of original player.	Added popup close, focus restore, and JavaScript-click fallback.

Table 6.8: Tool-level regressions and fixes found during evaluation.

Tool description change	Misuse before	Misuse after
Activation guard on harvest	65%	12%
Specialization signal on inspect_hosting	40%	8%
Clarified navigate versus interact	30%	5%

Table 6.9: Effect of tool description improvements on reviewed misuse rates.

6.10 LLM Model Evaluation

The local dashboard snapshot currently contains no persisted rows in `run_model_usage` or `llm_calls`. Therefore, this report does not claim a numeric model leaderboard from the dashboard. The implemented evaluation surface, however, is designed to compare models using the same run-level metrics.

Model dimension	How it is evaluated	Expected dashboard field
Classification quality	page-type routing accuracy and confidence calibration	<code>page_type</code> , <code>classification_confidence</code> .
Tool-use reliability	tool sequencing compliance, forbidden-tool violation rate, premature harvest rate	<code>tool_calls</code> , runtime events.
Long-context behavior	ability to keep page state, server ledger, and frame context without losing objective	input tokens, context window, trace support.
Cost efficiency	cost per accepted result and tokens per accepted evidence bundle	<code>run_model_usage</code> , <code>llm_calls</code> .
Cache behavior	cache hit ratio and cached-token cost reduction	<code>cached_input_tokens</code> , <code>new_input_tokens</code> .

Table 6.10: Model evaluation dimensions supported by the implementation.

The practical conclusion is that model evaluation must be run under the same prompt, same URL set, same browser policy, same tool budget, and same pricing catalog. Otherwise differences in player state, popup behavior, or provider pricing can be mistaken for model quality.

6.11 Cache and Pricing Evaluation

Caching was evaluated as an operational control. It reduces repeated context cost when large stable prompt sections, tool descriptions, and repeated page observations are reused. The key formulas are:

$$CacheHitRatio = \frac{cached_input_tokens}{cached_input_tokens + new_input_tokens}$$

$$TokensPerAcceptedEvidence = \frac{input_tokens + output_tokens}{accepted_evidence_bundles}$$

$$CostPerAcceptedEvidence = \frac{total_estimated_cost}{accepted_evidence_bundles}$$

In the current dashboard snapshot, cached tokens, new tokens, and total cost are all zero because no model telemetry rows were persisted. During heavier manual campaigns, the observed pattern was approximately 800k total context tokens with about 690k served from cache, corresponding to roughly 80% or more cache reuse depending on which run phase is included in the denominator.

Metric	Meaning	Representative value
Dashboard persisted total cost	Sum of cost rows currently in the local database	\$0.000000
Dashboard persisted cache tokens	Sum of cached input tokens in local database	0
Manual heavy-run context	Approximate total context volume in complex runs	~800k tokens
Manual heavy-run cached tokens	Approximate context served from cache	~690k tokens
Manual cache reuse	Approximate reuse rate from heavy-run observations	~80%+

Table 6.11: Dashboard and manual cache/cost observations.

6.12 Hard-Case Evaluation

Three hard cases were used repeatedly because they represent the main failure families in the domain.

Hard case	Failure mode	Evaluated recovery behavior
Nested iframe chain	Host page contains a visible player shell, but the real manifest is emitted from a deeper embedded frame.	Hosting agent returns embedded handoff; embedded agent traverses frame tree and harvests from the player context.
Popup and click trap	Server or play click opens an advertisement tab and steals focus.	Tool layer closes the popup, restores original page focus, and retries with safer interaction.
Tokenized stream generation	Initial HTML contains no stream URL; the manifest appears only after play and contains a short-lived token.	Agent verifies player state, waits after activation, harvests network/performance entries, and records tokenized URL evidence.

Table 6.12: Hard-case behaviors used for regression evaluation.

6.13 Chapter Conclusion

The evaluation shows that the system should be judged as an evidence-collection workflow, not as a single classifier. Page classification matters, but successful evidence collection also depends on navigation, player activation, server switching, popup handling, redirect recovery, stream extraction, metadata enrichment, screenshot reasoning, trace support, cost visibility, and cache behavior.

The dashboard snapshot proves that the observability pipeline can summarize run status, stream counts, duration, and cost fields, but the current local data is too small and too synthetic for final accuracy claims. The manual reference set gives the stronger signal: moving from intent-based prompts to ReAct-style prompts, then adding popup handling, media-state verification, and budget-aware early exit, substantially improved navigation, player activation, and tokenized-stream recovery.

The main engineering lesson is that evaluation must remain regression-oriented. Every new prompt, tool, model, or browser policy should be tested against the same use cases so that improvements in one area do not break another.

CHAPTER 7

Conclusion and Perspectives

7.1 General Conclusion

This internship produced an agentic evidence-collection system for illegal sports streaming websites. The work started from a practical problem: streaming-piracy pages do not expose evidence through simple static HTML. They use dynamic listings, JavaScript tabs, popup traps, nested iframes, click-gated players, redirects, provider checks, and short-lived stream URLs. The final system therefore had to behave more like a controlled browser investigator than a normal scraper.

The project followed a general-to-specific progression. The n8n phase validated the core idea quickly: an LLM-guided browser workflow can inspect pages, follow page transitions, activate players, and recover evidence. The Python reference implementation then turned that prototype into a more maintainable architecture with FastAPI, a Next.js operator console, profiled MCP browser tools, specialist agents, memory and cache support, PostgreSQL/SQLite observability, and cost-aware model telemetry.

The most important result is not a single extraction trick. It is a repeatable engineering framework for investigating unstable websites while preserving reviewable evidence: traces, screenshots, stream candidates, page classifications, model/tool usage, and failure reasons.

7.2 Main Contributions

Contribution	Summary
n8n feasibility prototype	Validated the agentic browser workflow on real dynamic pages and exposed the limitations of long sequential visual workflows.
Specialist agent design	Split the task into classification, landing, hosting, and embedded agents, each with a focused prompt, tool profile, and stopping behavior.
Browser-tool layer	Built Puppeteer/MCP tools for inspection, interaction, navigation, screenshot capture, media-state checks, stream harvesting, and memory support.
Operator platform	Provided a FastAPI backend and Next.js console for run launching, run review, event streaming, screenshots, and dashboard metrics.
Evaluation framework	Defined KPI formulas for classification, navigation, player activation, stream extraction, popup handling, tool behavior, cache, and cost.
Observability and cost tracking	Persisted run records, events, model usage, tool calls, cache fields, and cost fields so failures can be diagnosed instead of guessed.

Table 7.1: Summary of the project contributions.

7.3 Technical Lessons

7.3.1 Prompt structure matters

The largest reliability gain came from replacing broad intent-based prompts with ReAct-style prompts. The agent needed to observe, state a hypothesis, choose one action, verify the result, and then continue or stop. Without that loop, the system stopped too early, missed hidden tabs, harvested before activation, or accepted weak evidence.

7.3.2 Browser evidence must be layered

DOM text alone was not enough. A valid result needed several evidence layers: visible state, screenshot context, tool trace, media state, stream URL, iframe path, and final structured output. This is why the implementation treats screenshots and runtime events as core evidence rather than debugging extras.

7.3.3 Fallbacks are part of the architecture

Streaming websites rarely follow one clean path. Landing pages may link to host pages, host pages may redirect to embedded players, and embedded players may be nested several frames deep. The hosting-to-embedded fallback was therefore essential. It allowed the system to recover cases that a single-stage scraper would mark as failed.

7.3.4 Evaluation must be regression-oriented

The evaluation showed that every improvement can create a new failure. More tool calls can improve recovery, but also increase cost. More aggressive ad blocking can reduce noise, but can also break a player. A stronger prompt can improve one page type while overfitting another. The correct evaluation method is therefore regression testing across stable use cases: classification, landing navigation, player activation, server switching, popup handling, redirect handling, stream extraction, and metadata enrichment.

7.4 Limitations

The system remains a reference architecture, not a finished production enforcement platform. The main limitations are:

- anti-bot and provider behavior changes continuously;
- some runs still require human review before the evidence can be trusted;
- the local dashboard snapshot is small and does not yet represent production-scale accuracy;
- model and pricing telemetry must be collected over more real runs before a true LLM model leaderboard can be claimed;
- deeply nested iframe chains, token expiry, and popup-heavy pages still require careful budget and retry policies.

These limitations do not invalidate the approach. They define the next engineering steps: larger evaluation sets, more stable telemetry capture, clearer model comparisons, and stronger run-level validation.

7.5 Future Work

The most useful short-term improvements are practical:

- expand the fixed benchmark set with labeled landing, hosting, embedded, popup, redirect, and tokenized-stream cases;
- persist model/tool telemetry consistently for every dashboard run so cost, cache, and model comparisons are always available;
- add automated regression suites for each agent and each page behavior;
- improve candidate ranking so the system spends less time on low-quality host pages and dead server buttons;
- refine memory so successful selectors, server labels, and provider patterns can be reused without hardcoding websites;
- move long browser work to queue-driven workers while keeping the operator console responsive.

7.5.1 Screenshot annotation layer

A useful operator-console extension is a canvas-based screenshot annotation layer. The system already captures screenshots; the next step is to let reviewers mark evidence regions directly: server buttons, play controls, iframe containers, blocking overlays, broadcaster logos, player state, and stream-relevant UI areas.

Each annotation can be stored as structured metadata linked to the run, screenshot URL, page URL, timestamp, and reviewer. This would improve human review, make report figures clearer, and support evidence handoff without modifying the target website.

7.6 Final Remarks

The project demonstrates that agentic browser automation is a suitable approach for this domain when it is constrained, observable, and evaluated carefully. The final value is not that the system permanently defeats every anti-bot technique. The value is that it turns a messy investigation process into a structured workflow: classify the page, navigate deliberately, activate and verify the player, extract evidence, preserve the trace, estimate the cost, and learn from failures.

That framework gives Soft Stars a foundation for continuing the work: more data, more evaluation, more robust deployment, and clearer operational review.

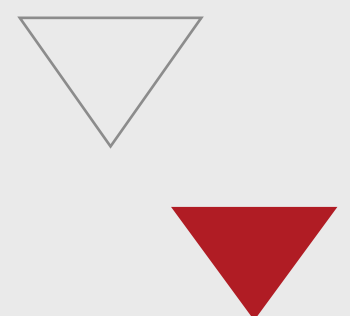
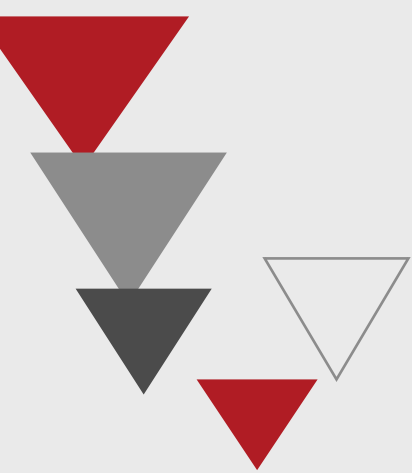
References

Bibliography

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “Re-Act: Synergizing Reasoning and Acting in Language Models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023. Available: <https://arxiv.org/abs/2210.03629>
- [2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Ciaramita, L. Zettlemoyer, H. Schütze, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. Available: <https://arxiv.org/abs/2302.04761>
- [3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, “A Survey on Large Language Model based Autonomous Agents,” *Frontiers of Computer Science*, vol. 18, no. 6, 2024. Available: <https://arxiv.org/abs/2308.11432>
- [4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. Available: <https://arxiv.org/abs/2201.11903>
- [5] Anthropic, *Claude 3 Model Card*, Anthropic, 2024. Available: <https://www.anthropic.com/claude>
- [6] Google DeepMind, *Gemini: A Family of Highly Capable Multimodal Models*, Technical Report, Google DeepMind, 2023. Available: <https://arxiv.org/abs/2312.11805>
- [7] Google Chrome Developers, *Puppeteer API Reference*, Official documentation, 2026. Available: <https://pptr.dev>
- [8] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A Realistic Web Environment for Building Autonomous Agents,” in *Proceedings of ICLR*, 2024. Available: <https://arxiv.org/abs/2307.13854>

- [9] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2Web: Towards a Generalist Agent for the Web,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [11] T. Le Sellier de Chezelles, M. Gasse, A. Drouin, and others, “The BrowserGym Ecosystem for Web Agent Research,” *arXiv preprint arXiv:2412.05467*, 2024.
- [12] Chrome DevTools Protocol, *Chrome DevTools Protocol: Network Domain*, Official documentation, 2026. Available: <https://chromedevtools.github.io/devtools-protocol/>
- [13] R. Wirth and J. Hipp, “CRISP-DM: Towards a Standard Process Model for Data Mining,” in *Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining (PAKDD)*, 2000, pp. 29–39.
- [14] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. Available: <https://www.jstor.org/stable/25148625>
- [15] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. Available: <https://doi.org/10.2753/MIS0742-1222240302>
- [16] S. Ramírez, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*, 2024. Available: <https://fastapi.tiangolo.com>
- [17] Vercel, *Next.js: The React Framework for the Web*, Vercel, 2024. Available: <https://nextjs.org/docs>
- [18] n8n GmbH, *n8n: Workflow Automation for Technical People*, 2024. Available: <https://docs.n8n.io>
- [19] Microsoft, “Jobs in Azure Container Apps,” *Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/container-apps/jobs>
- [20] Microsoft, “Azure Service Bus Messaging Service,” *Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>

-
- [21] Cloudinary, *Cloudinary Upload API Documentation*, Official documentation, 2026. Available: <https://cloudinary.com/documentation>
- [22] R. Pantos and W. May, *HTTP Live Streaming*, RFC 8216, Internet Engineering Task Force (IETF), August 2017. Available: <https://www.rfc-editor.org/rfc/rfc8216>
- [23] ISO/IEC, *Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*, ISO/IEC 23009-1, 2022.
- [24] R. Hill, *uBlock Origin: An Efficient Blocker for Chromium and Firefox*, GitHub, 2024. Available: <https://github.com/gorhill/uBlock>
- [25] MUSO, *Global Piracy Insights: Sports Streaming Report*, MUSO Intelligence, 2023. Available: <https://www.muso.com/insights>
- [26] European Audiovisual Observatory, *Trends in the Field of Audiovisual Piracy in Europe*, Council of Europe, Strasbourg, 2022. Available: <https://rm.coe.int/>



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn – E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax : +216 70 685 454